

GOVERNMENT OF KERALA
DEPARTMENT OF TECHNICAL EDUCATION
RAJIV GANDHI INSTITUTE OF TECHNOLOGY
(GOVT. ENGINEERING COLLEGE)
KOTTAYAM - 686501



RECORD BOOK

GOVERNMENT OF KERALA
DEPARTMENT OF TECHNICAL EDUCATION
RAJIV GANDHI INSTITUTE OF TECHNOLOGY
(GOVT. ENGINEERING COLLEGE)
KOTTAYAM - 686501



20MCA241 DATA SCIENCE LAB

Name: Arya Manoj
Branch: Master of Computer Applications
Semester: 3
Roll No: 12

CERTIFIED BONAFIDE RECORD WORK DONE BY

Reg No.

STAFF IN CHARGE

INTERNAL EXAMINER

EXTERNAL EXAMINER

Contents

Assignment 1 Review of Python Programming	1
Assignment 2 Vectorized Computations using NumPy	9
Part A: NumPy Fundamentals	9
Part B: Matrix Computations	16
Part C: Matrix Functions	23
Assignment 3 Data Manipulation using Pandas	29
Part A: Creating a DataFrame	29
Part B: Exploring the DataFrame	34
Part C: Reading Data	37
Part D: Creating New Data	42
Part E: Updating Data	46
Part F: Deleting Data	49
Part G: Filtering and Sorting	52
Part H: Saving, Loading and Missing Values	55
Assignment 4 Exploratory Data Analysis (EDA),Data Preprocessing, and Visualization	58

Assignment 1

Review of Python Programming

1. Write a function that accepts two numbers and performs arithmetic, relational and logical operations. Display the results along with the data types of the operands.

```
1 def number_operations(a,b):
2     print("Types:",type(a),type(b))
3     print("Addition:",a+b)
4     print("Subtraction:",a-b)
5     print("Multiplication:",a*b)
6     print("Division:",a/b)
7     print("Floor Division:",a//b)
8     print("Modulus:",a%b)
9     print("Power:",a**b)
10    print("a>b:",a>b)
11    print("a==b:",a==b)
12    print("(a>b) and (b>0):",(a>b) and (b>0))
13
14 number_operations(10,3)
```

Output

Types: <class 'int'> <class 'int'>

Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulus: 1
Power: 1000

a>b: True
a==b: False
(a>b) and (b>0): True

2. Write a function that accepts an integer and determines whether it is positive, negative or zero, and whether it is even or odd.

```
1 def analyze_number(n):
2     if n>0: sign="Positive"
3     elif n<0: sign="Negative"
4     else: sign="Zero"
5     parity="Even" if n%2==0 else "Odd"
6     print(sign,parity)
7
8 analyze_number(15)
```

Output

Positive Odd

3. Write a function that accepts n and computes (i) sum of first n natural numbers (ii) factorial (iii) all prime numbers $j \leq n$.

```
1 import math
2 def summary(n):
3     print("Sum:",sum(range(1,n+1)))
4     f=1
5     for i in range(2,n+1): f*=i
6     print("Factorial:",f)
7     primes=[]
8     for x in range(2,n+1):
9         ok=True
10        for d in range(2,int(math.sqrt(x))+1):
11            if x%d==0:
12                ok=False;break
13        if ok: primes.append(x)
14    print("Prime numbers till",n,":",primes)
15 summary(10)
```

Output

Sum: 55

Factorial: 3628800

Prime numbers till 10 : [2, 3, 5, 7]

4. Write a function that accepts a string and counts vowels/consonants, checks palindrome, reverses it and counts character frequencies.

```
1 from collections import Counter
2 def string_info(s):
3     vowels=sum(c.lower() in "aeiou" for c in s)
4     consonants=sum(c.isalpha() and c.lower() not in "aeiou" for c in s)
5     print("Vowels:",vowels)
6     print("Consonants:",consonants)
7     print("Palindrome:",s==s[::-1])
8     print("Reverse:",s[::-1])
9     print(Counter(s))
10 string_info("madam")
```

Output

```
Vowels: 2
Consonants: 3
Palindrome: True
Reverse: madam
Counter({'m': 2, 'a': 2, 'd': 1})
```

5. Write a function that accepts a list of numbers and returns the maximum, minimum, average, median, second largest element, and the list after removing duplicate elements.

```
1 import statistics
2 def list_stats(lst):
3     print("Maximum element:",max(lst))
4     print("Minimum element:",min(lst))
5     print("Average:",sum(lst)/len(lst))
6     print("Median:",statistics.median(lst))
7     print("Sorted List after removing duplicate elements:",sorted(set(lst)))
8     print("Second Largest element:",sorted(lst)[-2])
9 list_stats([5,2,7,2,8,8,9])
```

Output

```
Maximum element: 9
Minimum element: 2
Average: 5.857142857142857
Median: 7
Sorted List after removing duplicate elements: [2, 5, 7, 8, 9]
Second Largest element: 8
```

6. Write a function that accepts two lists and returns the common elements, union of the elements, and elements present only in the first list.

```
1 def compare_lists(a,b):
2     print("Common:",list(set(a)&set(b)))
3     print("Union:",list(set(a)|set(b)))
4     print("Only first:",list(set(a)-set(b)))
5 compare_lists([1,2,3],[2,3,4])
```

Output

```
Common: [2, 3]
Union: [1, 2, 3, 4]
Only first: [1]
```

7. Write a function that accepts a sentence and returns a dictionary containing frequency of each word.

```
1 from collections import Counter
2 def word_freq(text):
3     print(dict(Counter(text.lower().split())))
4 word_freq("this is a test this is easy")
```

Output

```
{'this': 2, 'is': 2, 'a': 1, 'test': 1, 'easy': 1}
```

8. Write a recursive function to compute the Fibonacci sequence up to n terms.

```
1 def fib(n):
2     if n<=1: return n
3     return fib(n-1)+fib(n-2)
4 print([fib(i) for i in range(8)])
```

Output

```
[0, 1, 1, 2, 3, 5, 8, 13]
```

9. Write a higher-order function that accepts another function and a list of numbers, then applies the function to every element of the list. Use this function to compute the square, cube, and square root of all elements.

```
1 import math
2 def apply(fun, lst):
3     return [fun(x) for x in lst]
4 nums=[1,4,9]
5 print(apply(lambda x:x*x,nums))
6 print(apply(lambda x:x**3,nums))
7 print(apply(math.sqrt,nums))
```

Output

```
[1, 16, 81]
[1, 64, 729]
[1.0, 2.0, 3.0]
```

10. Write a function that accepts the marks of several students and returns the highest, lowest, average marks, and the corresponding letter grades.

```
1 def grades(marks):
2     avg=sum(marks)/len(marks)
3     print("Maximum mark: ",max(marks))
4     print("Minimum mark: ",min(marks))
5     print("Average mark: ",avg)
6     print("Grades: ")
7     for m in marks:
8         g='A' if m>=90 else 'B' if m>=75 else 'C' if m>=60 else 'D'
9         print(m,g)
10 grades([95,81,67,55])
```

Output

```
Maximum mark: 95
Minimum mark: 55
Average mark: 74.5
Grades:
95 A
81 B
67 C
55 D
```

11. Write a function that reads integers from a text file and writes another file containing only the even numbers arranged in ascending order. Handle all possible exceptions.

```
1 with open("numbers.txt","w") as f:
2     f.write("5 2 8 9 12 6")
3 try:
4     with open("numbers.txt") as f:
5         nums=list(map(int,f.read().split()))
6         even=sorted([x for x in nums if x%2==0])
7         with open("even.txt","w") as f:
8             f.write(" ".join(map(str,even)))
9         print(even)
10 except Exception as e:
11     print(e)
```

Output

[2, 6, 8, 12]

12. Create a class Student with the attributes Roll Number, Name, and Marks of three subjects. Implement methods to compute the total and average, determine the grade, and display the student's details.

```
1 class Student:
2     def __init__(self,r,n,m):
3         self.roll=r;self.name=n;self.marks=m
4     def total(self): return sum(self.marks)
5     def average(self): return self.total()/len(self.marks)
6     def grade(self):
7         a=self.average()
8         return 'A' if a>=90 else 'B' if a>=75 else 'C'
9     def display(self):
10         print(self.roll,self.name,self.total(),self.average(),self.grade())
11 s=Student(1,"Alice",[90,85,95]);s.display()
```

Output

1 Alice 270 90.0 A

13. Create a class BankAccount with methods for deposit, withdrawal, balance enquiry, and transaction history. Ensure that withdrawal is allowed only if sufficient balance exists.

```
1 class BankAccount:
2     def __init__(self,b=0):
3         self.balance=b;self.history=[]
4     def deposit(self,a):
5         self.balance+=a;self.history.append(("Deposit",a))
6     def withdraw(self,a):
7         if a<=self.balance:
8             self.balance-=a;self.history.append(("Withdraw",a))
9         else: print("Insufficient balance")
10    def show(self):
11        print(self.balance,self.history)
12 acc=BankAccount(1000)
13 acc.deposit(500);acc.withdraw(700);acc.show()
```

Output

```
800 [('Deposit', 500), ('Withdraw', 700)]
```

14. Create a base class Employee and derive the classes Faculty and Staff. Override a method to compute the monthly salary according to different rules for each category.

```
1 class Employee:
2     def salary(self): return 0
3 class Faculty(Employee):
4     def salary(self): return 80000
5 class Staff(Employee):
6     def salary(self): return 40000
7 for e in [Faculty(),Staff()]:
8     print(type(e).__name__,e.salary())
```

Output

```
Faculty 80000
Staff 40000
```

15. Create a class Shape with a method area(). Derive the classes Circle, Rectangle, and Triangle, and override the area() method appropriately. Write a program that computes the area of different shapes using polymorphism.

```
1 import math
2 class Shape:
3     def area(self): pass
4 class Circle(Shape):
5     def __init__(self,r): self.r=r
6     def area(self): return math.pi*self.r*self.r
7 class Rectangle(Shape):
8     def __init__(self,l,b): self.l=l;self.b=b
9     def area(self): return self.l*self.b
10 for s in [Circle(2),Rectangle(3,4)]:
11     print(s.area())
```

Output

```
12.566370614359172
12
```

16. Create a menu-driven program that allows the user to execute any of the above problems repeatedly until the user chooses to exit.

```
1 def menu():
2     while True:
3         print("1.Number Analysis 2.String Reverse 3.Exit")
4         ch=input("Choice: ")
5         if ch=="1":
6             analyze_number(int(input("Number: ")))
7         elif ch=="2":
8             print(input("String: ")[::-1])
9         else:
10            break
11 menu()
```

Output

```
1.Number Analysis 2.String Reverse 3.Exit
Choice: 1
Number: 34
Positive Even
1.Number Analysis 2.String Reverse 3.Exit
Choice: 2
String: hello
olleh
1.Number Analysis 2.String Reverse 3.Exit
Choice: 3
```

Assignment 2

Vectorized Computations using NumPy

Part A: NumPy Fundamentals

1. Read two positive integers m and n from the user.

```
1 import numpy as np
2 m = int(input("Enter a positive integer for m: "))
3 n = int(input("Enter a positive integer for n: "))
4 print(f"m: {m}, n: {n}")
```

Output

```
Enter a positive integer for m: 5
Enter a positive integer for n: 4
m: 5, n: 4
```

2. Create a matrix U of shape (m, n) using user input.

```
1 print(f"Creating matrix U of shape ({m}, {n}) with random integers for ↵
    demonstration.")
2 U = np.random.randint(0, 10, size=(m, n))
3 display(U)
```

Output

```
Creating matrix U of shape (5, 4) with random integers for demonstration.
array([[4, 3, 9, 4],
       [1, 2, 6, 8],
       [0, 8, 0, 2],
       [5, 5, 8, 1],
       [5, 5, 9, 5]])
```

3. Display the shape, number of dimensions, size, and data type of U.

```
1 print(f"Shape of U: {U.shape}")
2 print(f"Number of dimensions of U: {U.ndim}")
3 print(f"Size of U (total elements): {U.size}")
4 print(f>Data type of U: {U.dtype}")
```

Output

Shape of U: (5, 4)
Number of dimensions of U: 2
Size of U (total elements): 20
Data type of U: int64

4. Compute and display the transpose of U.

```
1 U_transpose = U.T
2 print("Transpose of U:")
3 display(U_transpose)
```

Output

Transpose of U:
array([[4, 1, 0, 5, 5],
 [3, 2, 8, 5, 5],
 [9, 6, 0, 8, 9],
 [4, 8, 2, 1, 5]])

5. Create the following matrices:

- (a) A matrix of zeros having shape (m, n).
- (b) A matrix of ones having shape (m, n).
- (c) A matrix of random values in [0, 1] having shape (m, n).
- (d) An identity matrix of order n.

```
1 zeros_matrix = np.zeros((m, n))
2 print("Matrix of zeros:")
3 display(zeros_matrix)
4
5 ones_matrix = np.ones((m, n))
6 print("Matrix of ones:")
7 display(ones_matrix)
8
9 random_matrix = np.random.rand(m, n)
10 print("Matrix of random values in [0, 1]:")
11 display(random_matrix)
12
13 identity_matrix = np.identity(n)
14 print(f"Identity matrix of order {n}:")
15 display(identity_matrix)
```

Output

Matrix of zeros:

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

Matrix of ones:

```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

Matrix of random values in [0, 1]:

```
array([[0.8214219 , 0.32507843, 0.58176022, 0.63640828],
       [0.87927504, 0.22144489, 0.66054254, 0.69814874],
       [0.91181467, 0.37390159, 0.99108053, 0.60309063],
       [0.83405101, 0.71142373, 0.72704089, 0.69635179],
       [0.85104201, 0.88222488, 0.07747413, 0.22241173]])
```

Identity matrix of order 4:

```
array([[1., 0., 0., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 0., 0., 1.]])
```

6. Perform the following operations on U.
- (a) Find the maximum and minimum values.
 - (b) Find the mean, median, variance and standard deviation.
 - (c) Find the row-wise sum.
 - (d) Find the column-wise sum.
 - (e) Find the overall sum.

```
1 print(f"Maximum value in U: {U.max()}")
2 print(f"Minimum value in U: {U.min()}")
3
4 print(f"Mean of U: {U.mean()}")
5 print(f"Median of U: {np.median(U)}")
6 print(f"Variance of U: {U.var()}")
7 print(f"Standard deviation of U: {U.std()}")
8
9 row_sums = U.sum(axis=1)
10 print("Row-wise sums of U:")
11 display(row_sums)
12
13 col_sums = U.sum(axis=0)
14 print("Column-wise sums of U:")
15 display(col_sums)
16
17 overall_sum = U.sum()
18 print(f"Overall sum of U: {overall_sum}")
```

Output

```
Maximum value in U: 9
Minimum value in U: 0
Mean of U: 4.5
Median of U: 5.0
Variance of U: 8.05
Standard deviation of U: 2.8372521918222215
Row-wise sums of U:

array([20, 17, 10, 19, 24])

Column-wise sums of U:

array([15, 23, 32, 20])

Overall sum of U: 90
```

7. Reshape U into any valid dimensions and display the result.

```
1 original_size = U.size
2 if original_size % m == 0:
3     reshaped_U = U.reshape(m, original_size // m)
4     print(f"Reshaped U to ({m}, {original_size // m}):")
5 elif original_size % n == 0:
6     reshaped_U = U.reshape(n, original_size // n)
7     print(f"Reshaped U to ({n}, {original_size // n}):")
8 else:
9     reshaped_U = U.reshape(1, original_size)
10    print(f"Reshaped U to (1, {original_size}) as other simple reshapes were ↵
        not direct:")
11 display(reshaped_U)
```

Output

```
Reshaped U to (5, 4):
array([[4, 3, 9, 4],
       [1, 2, 6, 8],
       [0, 8, 0, 2],
       [5, 5, 8, 1],
       [5, 5, 9, 5]])
```

8. Flatten the matrix into a one-dimensional array.

```
1 flattened_U = U.flatten()
2 print("Flattened U into a one-dimensional array:")
3 display(flattened_U)
```

Output

```
Flattened U into a one-dimensional array:
array([4, 3, 9, 4, 1, 2, 6, 8, 0, 8, 0, 2, 5, 5, 8, 1, 5, 5, 9, 5])
```

9. Extract the first row, last row, first column and last column.

```
1 first_row = U[0, :]  
2 last_row = U[-1, :]  
3 first_column = U[:, 0]  
4 last_column = U[:, -1]  
5  
6 print("First row of U:")  
7 display(first_row)  
8 print("Last row of U:")  
9 display(last_row)  
10 print("First column of U:")  
11 display(first_column)  
12 print("Last column of U:")  
13 display(last_column)
```

Output

First row of U:
array([4, 3, 9, 4])

Last row of U:
array([5, 5, 9, 5])

First column of U:
array([4, 1, 0, 5, 5])

Last column of U:
array([4, 8, 2, 1, 5])

10. Concatenate U with itself horizontally and vertically.

```
1 U_h_concat = np.concatenate((U, U), axis=1)
2 print("U concatenated with itself horizontally:")
3 display(U_h_concat)
4
5 U_v_concat = np.concatenate((U, U), axis=0)
6 print("U concatenated with itself vertically:")
7 display(U_v_concat)
```

Output

U concatenated with itself horizontally:

```
array([[4, 3, 9, 4, 4, 3, 9, 4],
       [1, 2, 6, 8, 1, 2, 6, 8],
       [0, 8, 0, 2, 0, 8, 0, 2],
       [5, 5, 8, 1, 5, 5, 8, 1],
       [5, 5, 9, 5, 5, 5, 9, 5]])
```

U concatenated with itself vertically:

```
array([[4, 3, 9, 4],
       [1, 2, 6, 8],
       [0, 8, 0, 2],
       [5, 5, 8, 1],
       [5, 5, 9, 5],
       [4, 3, 9, 4],
       [1, 2, 6, 8],
       [0, 8, 0, 2],
       [5, 5, 8, 1],
       [5, 5, 9, 5]])
```

Part B: Matrix Computations

Assume that $X = U^T$, where X has shape (n, m) .

```
1 X = U.T
2 print("Matrix X (U.T):")
3 display(X)
4 print(f"Shape of X: {X.shape}")
```

Output

```
Matrix X (U.T):
array([[4, 1, 0, 5, 5],
       [3, 2, 8, 5, 5],
       [9, 6, 0, 8, 9],
       [4, 8, 2, 1, 5]])
Shape of X: (4, 5)
```

1. Read a positive integer p .

```
1 p = int(input("Enter a positive integer for p: "))
2 print(f"p: {p}")
```

Output

```
Enter a positive integer for p: 5
p: 5
```

2. Create the following matrices with random values in $[0, 1]$.

- (a) W of shape (p, n)
- (b) B of shape $(p, 1)$
- (c) V of shape $(1, p)$
- (d) C of shape $(1, 1)$

```
1 W = np.random.rand(p, n)
2 print(f"Matrix W of shape {W.shape}:")
3 display(W)
4
5 B = np.random.rand(p, 1)
6 print(f"Matrix B of shape {B.shape}:")
7 display(B)
8
9 V = np.random.rand(1, p)
10 print(f"Matrix V of shape {V.shape}:")
11 display(V)
12
13 C = np.random.rand(1, 1)
14 print(f"Matrix C of shape {C.shape}:")
15 display(C)
```

Output

Matrix W of shape $(5, 4)$:

```
array([[0.90767639, 0.49775039, 0.16168126, 0.34751807],
       [0.51473445, 0.69525348, 0.26589883, 0.1064297 ],
       [0.25962804, 0.98444123, 0.82715218, 0.21855541],
       [0.95772854, 0.80888915, 0.63143767, 0.33525386],
       [0.91212781, 0.18852865, 0.8439743 , 0.67108294]])
```

Matrix B of shape $(5, 1)$:

```
array([[0.33247552],
       [0.35077103],
       [0.81627265],
       [0.44635118],
       [0.90417416]])
```

Matrix V of shape $(1, 5)$:

```
array([[0.9050105 , 0.44568056, 0.3795386 , 0.23665197, 0.79827955]])
```

Matrix C of shape $(1, 1)$:

```
array([[0.41984018]])
```

3. Compute $M1=WX+B$ using NumPy matrix multiplication.

```
1 M1 = np.dot(W, X) + B
2 print("Matrix M1 (WX + B):")
3 display(M1)
4 print(f"Shape of M1: {M1.shape}")
```

Output

Matrix M1 (WX + B):

```
array([[ 8.3016358 ,  5.98588476,  5.00951474,  9.0005775 , 10.55233103],
       [ 7.31427752,  4.70284299,  6.12565828,  8.63433101,  9.32594862],
       [13.1266998 ,  9.75613953,  9.12891333, 13.8723919 , 15.57376572],
       [13.72788724,  9.49251489,  7.58797213, 14.66619486, 16.63864795],
       [15.39837179, 12.62586855,  3.7545692 , 13.83033379, 17.35863983]])
```

Shape of M1: (5, 5)

4. Perform element-wise multiplication between two compatible matrices.

```
1 compatible_matrix_elem_mul = np.random.rand(*M1.shape)
2 print("Compatible matrix for element-wise multiplication:")
3 display(compatible_matrix_elem_mul)
4
5 elem_wise_mul = M1 * compatible_matrix_elem_mul
6 print("Element-wise multiplication of M1 and a compatible matrix:")
7 display(elem_wise_mul)
```

Output

Compatible matrix for element-wise multiplication:

```
array([[0.93834879, 0.26919031, 0.06327299, 0.98898002, 0.9568278 ],
       [0.73568196, 0.9502343 , 0.33751805, 0.07710346, 0.58188073],
       [0.91838691, 0.81894538, 0.94152954, 0.75182748, 0.33275587],
       [0.35471684, 0.14368075, 0.5066035 , 0.38297034, 0.87376132],
       [0.98800331, 0.05919566, 0.74880789, 0.2235382 , 0.83316629]])
```

Element-wise multiplication of M1 and a compatible matrix:

```
array([[ 7.78982993,  1.61134217,  0.31696698,  8.90139128, 10.09676366],
       [ 5.38098203,  4.46880273,  2.06752023,  0.66573682,  5.42658984],
       [12.05538928,  7.9897454 ,  8.59514153, 10.42964545,  5.18226201],
       [ 4.86951279,  1.36389163,  3.84409327,  5.61671767, 14.53820703],
       [15.21364236,  0.7473966 ,  2.81145105,  3.09160787, 14.46263347]])
```

5. Perform element-wise division between two compatible matrices.

```
1 compatible_matrix_elem_div = np.random.rand(*M1.shape) + 0.1
2 print("Compatible matrix for element-wise division:")
3 display(compatible_matrix_elem_div)
4
5 elem_wise_div = M1 / compatible_matrix_elem_div
6 print("Element-wise division of M1 and a compatible matrix:")
7 display(elem_wise_div)
```

Output

Compatible matrix for element-wise division:

```
array([[0.46555078, 0.26740044, 0.40074489, 0.88788101, 1.08472794],
       [0.39399874, 0.11792294, 0.85629168, 0.16819961, 0.96865598],
       [0.59406878, 0.91792504, 0.15501358, 1.06256281, 1.01986104],
       [0.26484415, 0.8492604 , 0.92413077, 0.52990399, 0.38623189],
       [0.32068393, 0.88439486, 0.11958792, 0.63529725, 0.41384158]])
```

Element-wise division of M1 and a compatible matrix:

```
array([[17.83185895, 22.38547103, 12.50050808, 10.1371438 ,  9.72809003],
       [18.56421545, 39.88064665,  7.15370525, 51.333836  ,  9.62772008],
       [22.09626282, 10.62847088, 58.89105584, 13.05559711, 15.27047814],
       [51.83383299, 11.17739016,  8.21092896, 27.67707932, 43.07942521],
       [48.01728484, 14.27627983, 31.39589127, 21.76986249, 41.94513231]])
```

6. Apply the following mathematical functions on M1.

- (a) Absolute value
- (b) Square
- (c) Square root
- (d) Exponential
- (e) Natural logarithm

```
1 abs_M1 = np.abs(M1)
2 print("Absolute value of M1:")
3 display(abs_M1)
4
5 square_M1 = np.square(M1)
6 print("Square of M1:")
7 display(square_M1)
8
9 sqrt_M1 = np.sqrt(np.abs(M1))
10 print("Square root of M1 (using absolute values to ensure real results):")
11 display(sqrt_M1)
12
13 exp_M1 = np.exp(M1)
14 print("Exponential of M1:")
15 display(exp_M1)
16
17 log_M1 = np.log(np.abs(M1) + 1e-9)
18 print("Natural logarithm of M1 (using absolute values and epsilon to ensure ↵
    real results):")
19 display(log_M1)
```

Output

Absolute value of M1:

```
array([[ 8.3016358 ,  5.98588476,  5.00951474,  9.0005775 , 10.55233103],
       [ 7.31427752,  4.70284299,  6.12565828,  8.63433101,  9.32594862],
       [13.1266998 ,  9.75613953,  9.12891333, 13.8723919 , 15.57376572],
       [13.72788724,  9.49251489,  7.58797213, 14.66619486, 16.63864795],
       [15.39837179, 12.62586855,  3.7545692 , 13.83033379, 17.35863983]])
```

Square of M1:

```
array([[ 68.91715702,  35.83081637,  25.09523792,  81.01039534,
        111.35169013],
       [ 53.49865557,  22.11673219,  37.52368939,  74.55167196,
        86.97331773],
       [172.31024764,  95.18225847,  83.3370586 , 192.44325697,
        242.54217855],
       [188.4548881 ,  90.10783885,  57.57732101, 215.09727153,
```

```

276.84460553],
[237.1098538 , 159.41255673, 14.09678991, 191.27813262,
301.3223767 ]])

```

Square root of M1 (using absolute values to ensure real results):

```

array([[2.88125594, 2.44660679, 2.23819453, 3.00009625, 3.24843517],
       [2.7044921 , 2.16860393, 2.47500672, 2.93842322, 3.05384162],
       [3.62307877, 3.12348196, 3.02140916, 3.724566 , 3.94636107],
       [3.70511636, 3.08099252, 2.7546274 , 3.82964683, 4.07904988],
       [3.92407592, 3.55328982, 1.93767108, 3.71891567, 4.1663701 ]])

```

Exponential of M1:

```

array([[4.03046005e+03, 3.97774301e+02, 1.49832011e+02, 8.10776481e+03,
       3.82665348e+04],
       [1.50158653e+03, 1.10260196e+02, 4.57445743e+02, 5.62137181e+03,
       1.12255603e+04],
       [5.02172919e+05, 1.72598716e+04, 9.21799961e+03, 1.05853020e+06,
       5.80230445e+06],
       [9.16105893e+05, 1.32601011e+04, 1.97430581e+03, 2.34124841e+06,
       1.68295688e+07],
       [4.86886686e+06, 3.04330193e+05, 4.27158141e+01, 1.01493364e+06,
       3.45749444e+07]])

```

Natural logarithm of M1 (using absolute values and epsilon to ensure real results):

```

array([[2.11645258, 1.78940416, 1.61133905, 2.19728874, 2.35634679],
       [1.98982826, 1.54816722, 1.81248623, 2.15574623, 2.23280069],
       [2.57464831, 2.27789678, 2.21144667, 2.62990067, 2.74558781],
       [2.61942933, 2.25050358, 2.02656438, 2.68554518, 2.81172818],
       [2.73426178, 2.53574777, 1.32297355, 2.62686428, 2.85409036]])

```

7. Compute $M2=VM1+C$ using vectorized matrix multiplication.

```
1 M2 = np.dot(V, M1) + C
2 print("Matrix M2 (VM1 + C):")
3 display(M2)
4 print(f"Shape of M2: {M2.shape}")
```

Output

```
Matrix M2 (VM1 + C):
array([[31.71576517, 23.96132099, 15.94126977, 32.18997553, 37.8316658 ]])
Shape of M2: (1, 5)
```

8. Display the shapes of all intermediate matrices.

```
1 print(f"Shape of U: {U.shape}")
2 print(f"Shape of X: {X.shape}")
3 print(f"Shape of W: {W.shape}")
4 print(f"Shape of B: {B.shape}")
5 print(f"Shape of V: {V.shape}")
6 print(f"Shape of C: {C.shape}")
7 print(f"Shape of M1: {M1.shape}")
8 print(f"Shape of M2: {M2.shape}")
```

Output

```
Shape of U: (5, 4)
Shape of X: (4, 5)
Shape of W: (5, 4)
Shape of B: (5, 1)
Shape of V: (1, 5)
Shape of C: (1, 1)
Shape of M1: (5, 5)
Shape of M2: (1, 5)
```

Part C: Matrix Functions

1. Write a function `relu(X)` that returns $f(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$

```
1 def relu(X):
2     """Implements the ReLU activation function."""
3     return np.maximum(0, X)
4 print("ReLU function defined.")
```

Output

ReLU function defined.

2. Write a function `sigmoid(X)` that returns $g(x) = \frac{1}{1+e^{-x}}$

```
1 def sigmoid(X):
2     """Implements the Sigmoid activation function."""
3     return 1 / (1 + np.exp(-X))
4 print("Sigmoid function defined.")
```

Output

Sigmoid function defined.

3. Compute $Z1=WX+B$ and apply the ReLU function $A1=relu(Z1)$

```
1 Z1 = np.dot(W, X) + B
2 print("Matrix Z1 (WX + B):")
3 display(Z1)
4 print(f"Shape of Z1: {Z1.shape}")
5
6 A1 = relu(Z1)
7 print("Matrix A1 (relu(Z1)):")
8 display(A1)
9 print(f"Shape of A1: {A1.shape}")
```

Output

Matrix Z1 (WX + B):

```
array([[ 8.3016358 ,  5.98588476,  5.00951474,  9.0005775 , 10.55233103],
       [ 7.31427752,  4.70284299,  6.12565828,  8.63433101,  9.32594862],
       [13.1266998 ,  9.75613953,  9.12891333, 13.8723919 , 15.57376572],
       [13.72788724,  9.49251489,  7.58797213, 14.66619486, 16.63864795],
       [15.39837179, 12.62586855,  3.7545692 , 13.83033379, 17.35863983]])
```

Shape of Z1: (5, 5)
Matrix A1 (relu(Z1)):

```
array([[ 8.3016358 ,  5.98588476,  5.00951474,  9.0005775 , 10.55233103],
       [ 7.31427752,  4.70284299,  6.12565828,  8.63433101,  9.32594862],
       [13.1266998 ,  9.75613953,  9.12891333, 13.8723919 , 15.57376572],
       [13.72788724,  9.49251489,  7.58797213, 14.66619486, 16.63864795],
       [15.39837179, 12.62586855,  3.7545692 , 13.83033379, 17.35863983]])
```

Shape of A1: (5, 5)

4. Compute $Z2=VA1+C$ and apply the Sigmoid function $A2=\text{sigmoid}(Z2)$

```
1 Z2 = np.dot(V, A1) + C
2 print("Matrix Z2 (V A1 + C):")
3 display(Z2)
4 print(f"Shape of Z2: {Z2.shape}")
5
6 A2 = sigmoid(Z2)
7 print("Matrix A2 (sigmoid(Z2)):")
8 display(A2)
9 print(f"Shape of A2: {A2.shape}")
```

Output

Matrix Z2 (V A1 + C):

```
array([[31.71576517, 23.96132099, 15.94126977, 32.18997553, 37.8316658 ]])
```

Shape of Z2: (1, 5)
Matrix A2 (sigmoid(Z2)):

```
array([[1.          , 1.          , 0.99999988, 1.          , 1.          ]])
```

Shape of A2: (1, 5)

5. Print

- (a) Z1
- (b) A1
- (c) Z2
- (d) A2

```
1 print("Z1:")
2 display(Z1)
3
4 print("A1:")
5 display(A1)
6
7 print("Z2:")
8 display(Z2)
9
10 print("A2:")
11 display(A2)
```

Output

Z1:

```
array([[ 8.3016358 ,  5.98588476,  5.00951474,  9.0005775 , 10.55233103],
       [ 7.31427752,  4.70284299,  6.12565828,  8.63433101,  9.32594862],
       [13.1266998 ,  9.75613953,  9.12891333, 13.8723919 , 15.57376572],
       [13.72788724,  9.49251489,  7.58797213, 14.66619486, 16.63864795],
       [15.39837179, 12.62586855,  3.7545692 , 13.83033379, 17.35863983]])
```

A1:

```
array([[ 8.3016358 ,  5.98588476,  5.00951474,  9.0005775 , 10.55233103],
       [ 7.31427752,  4.70284299,  6.12565828,  8.63433101,  9.32594862],
       [13.1266998 ,  9.75613953,  9.12891333, 13.8723919 , 15.57376572],
       [13.72788724,  9.49251489,  7.58797213, 14.66619486, 16.63864795],
       [15.39837179, 12.62586855,  3.7545692 , 13.83033379, 17.35863983]])
```

Z2:

```
array([[31.71576517, 23.96132099, 15.94126977, 32.18997553, 37.8316658 ]])
```

A2:

```
array([[1.          , 1.          , 0.99999988, 1.          , 1.          ]])
```

6. Repeat the above computations for 10 iterations using newly generated random matrices.

```
1 print("Repeating computations for 10 iterations with newly generated random ←
    matrices:")
2
3 for i in range(1, 11):
4     print(f"\n--- Iteration {i} ---")
5
6     W_iter = np.random.rand(p, n)
7     B_iter = np.random.rand(p, 1)
8     V_iter = np.random.rand(1, p)
9     C_iter = np.random.rand(1, 1)
10
11     Z1_iter = np.dot(W_iter, X) + B_iter
12     print("Z1 (iteration {i}):")
13     display(Z1_iter)
14
15     A1_iter = relu(Z1_iter)
16     print("A1 (iteration {i}):")
17     display(A1_iter)
18
19     Z2_iter = np.dot(V_iter, A1_iter) + C_iter
20     print("Z2 (iteration {i}):")
21     display(Z2_iter)
22
23     A2_iter = sigmoid(Z2_iter)
24     print("A2 (iteration {i}):")
25     display(A2_iter)
26
27 print("\nAll 10 iterations completed.")
```

Output

```
array([[1.          , 0.99999999, 0.99998489, 1.          , 1.          ]])
```

```
--- Iteration 8 ---
```

```
Z1 (iteration {i}):
```

```
array([[14.05945151, 13.05866439,  7.84896536, 12.79151933, 16.76040494],
       [14.95091997, 12.65693183,  6.0752224 , 13.79926692, 17.37317434],
       [13.31524567,  8.09675702,  7.37449361, 14.81196272, 15.74082192],
       [ 9.02078809,  7.57055343,  7.54460514,  9.34095795, 11.04938976],
       [ 8.16040501,  7.30312346,  2.51872439,  7.20348408,  9.52662513]])
```

```
A1 (iteration {i}):
```

```
array([[14.05945151, 13.05866439,  7.84896536, 12.79151933, 16.76040494],
```

```

[14.95091997, 12.65693183, 6.0752224 , 13.79926692, 17.37317434],
[13.31524567, 8.09675702, 7.37449361, 14.81196272, 15.74082192],
[ 9.02078809, 7.57055343, 7.54460514, 9.34095795, 11.04938976],
[ 8.16040501, 7.30312346, 2.51872439, 7.20348408, 9.52662513]]

```

Z2 (iteration {i}):

```
array([[24.32461681, 19.33917472, 15.21866767, 24.61746037, 29.07361057]])
```

A2 (iteration {i}):

```
array([[1.          , 1.          , 0.99999975, 1.          , 1.          ]])
```

--- Iteration 9 ---

Z1 (iteration {i}):

```

array([[10.10364684, 7.21550628, 8.9195175 , 11.71397088, 13.14496977],
[ 8.1875167 , 8.90979535, 4.30358496, 6.73188028, 9.962983  ],
[10.8431393 , 8.01456489, 8.43771299, 11.73193756, 13.01231075],
[15.51302523, 12.91353236, 3.15726285, 13.54341392, 17.13109774],
[11.15919748, 10.56195734, 8.32651467, 10.57368978, 13.44766118]])

```

A1 (iteration {i}):

```

array([[10.10364684, 7.21550628, 8.9195175 , 11.71397088, 13.14496977],
[ 8.1875167 , 8.90979535, 4.30358496, 6.73188028, 9.962983  ],
[10.8431393 , 8.01456489, 8.43771299, 11.73193756, 13.01231075],
[15.51302523, 12.91353236, 3.15726285, 13.54341392, 17.13109774],
[11.15919748, 10.56195734, 8.32651467, 10.57368978, 13.44766118]])

```

Z2 (iteration {i}):

```
array([[30.24383683, 24.86059051, 14.50188845, 28.86353448, 34.9689731 ]])
```

A2 (iteration {i}):

```
array([[1.          , 1.          , 0.9999995, 1.          , 1.          ]])
```

--- Iteration 10 ---

Z1 (iteration {i}):

```

array([[ 9.44831628,  9.59530735,  2.75610348,  7.37050835, 10.57117726],
[15.77950286, 14.1462149 ,  6.63735504, 14.1983346 , 18.39026249],

```

```
[ 3.83498941,  3.62810106,  3.75780275,  3.98509235,  5.06813604],
[ 7.77528389,  5.03909926,  3.0783733 ,  8.23700264,  9.20473992],
[10.36278008, 11.32586192, 10.00371982,  9.56449865, 13.11739617]])
```

A1 (iteration {i}):

```
array([[ 9.44831628,  9.59530735,  2.75610348,  7.37050835, 10.57117726],
       [15.77950286, 14.1462149 ,  6.63735504, 14.1983346 , 18.39026249],
       [ 3.83498941,  3.62810106,  3.75780275,  3.98509235,  5.06813604],
       [ 7.77528389,  5.03909926,  3.0783733 ,  8.23700264,  9.20473992],
       [10.36278008, 11.32586192, 10.00371982,  9.56449865, 13.11739617]])
```

Z2 (iteration {i}):

```
array([[32.43457802, 31.59619835, 18.97422916, 29.00775176, 38.49317727]])
```

A2 (iteration {i}):

```
array([[1.          , 1.          , 0.99999999, 1.          , 1.          ]])
```

Assignment 3

Vectorized Computations using NumPy

Part A: Creating a DataFrame

1. Initialize the NumPy random number generator using seed value 10.

```
1 import numpy as np
2 import pandas as pd
3
4 # Initialize the NumPy random number generator using seed value 10
5 np.random.seed(10)
6
7 print("NumPy and Pandas imported. Random seed initialized to 10.")
```

Output

NumPy and Pandas imported. Random seed initialized to 10.

2. Create a synthetic student dataset containing 1000 students with the following attributes.

- (a) ID
- (b) Age
- (c) Height
- (d) Weight
- (e) Gender
- (f) Department
- (g) CGPA
- (h) Semester
- (i) Hosteller
- (j) Grade

```
1 # 2. Create a synthetic student dataset
2
3 # Define the number of students
4 num_students = 1000
5
6 print(f"Creating a synthetic dataset for {num_students} students...")
7
8 # (a) ID: Unique IDs from 1 to num_students
9 student_ids = np.arange(1, num_students + 1)
10
11 # (b) Age: Random integers between 18 and 25
```

```

12 student_age = np.random.randint(18, 26, size=num_students)
13
14 # (c) Height: Random floats between 150 and 190 cm
15 student_height = np.random.uniform(150, 190, size=num_students).round(2)
16
17 # (d) Weight: Random floats between 50 and 90 kg
18 student_weight = np.random.uniform(50, 90, size=num_students).round(2)
19
20 # (e) Gender: Randomly choose 'Male' or 'Female'
21 genders = np.random.choice(['Male', 'Female'], size=num_students)
22
23 # (f) Department: Randomly choose from a list of departments
24 departments = np.random.choice(['CS', 'EE', 'ME', 'CE', 'EC', 'IT'], size←
    =num_students)
25
26 # (g) CGPA: Random floats between 6.0 and 10.0
27 cgpas = np.random.uniform(6.0, 10.0, size=num_students).round(2)
28
29 # (h) Semester: Random integers between 1 and 8
30 semesters = np.random.randint(1, 9, size=num_students)
31
32 # (i) Hosteller: Randomly choose 'Yes' or 'No'
33 hostellers = np.random.choice(['Yes', 'No'], size=num_students)
34
35 # (j) Grade: Assign based on CGPA
36 def assign_grade(cgpa):
37     if cgpa >= 9.0:
38         return 'A'
39     elif cgpa >= 8.0:
40         return 'B'
41     elif cgpa >= 7.0:
42         return 'C'
43     elif cgpa >= 6.0:
44         return 'D'
45     else:
46         return 'F'
47
48 grades = np.vectorize(assign_grade)(cgpas)
49
50 # Create a Pandas DataFrame
51 student_data = pd.DataFrame({
52     'ID': student_ids,
53     'Age': student_age,
54     'Height': student_height,
55     'Weight': student_weight,
56     'Gender': genders,
57     'Department': departments,
58     'CGPA': cgpas,
59     'Semester': semesters,

```

```

60     'Hosteller': hostellers,
61     'Grade': grades
62 })
63
64 print("\nSynthetic student dataset created successfully:")
65 display(student_data.head()) # Display the first 5 rows
66
67 print(f"\nDataset shape: {student_data.shape}")
68 print("Dataset info:")
69 display(student_data.info())

```

Output

Creating a synthetic dataset for 1000 students...

```

synthetic student dataset created successfully:

ID  Age  Height  Weight  Gender  Department  GPA  Semester  Hosteller  Grade
0   1   19   161.19  70.54  Female      IT   6.81      0         Yes      B
1   2   25   180.46  80.77  Female      EC   6.43      0         No       D
2   3   22   163.37  68.03  Female      EE   6.67      0         Yes      B
3   4   25   182.46  70.33  Male        CE   6.06      1         Yes      B
4   5   18   170.92  65.30  Female      IT   6.13      0         No       A

Dataset shape: (1000, 10)
Dataset info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   ID           1000 non-null    int64
1   Age          1000 non-null    int64
2   Height       1000 non-null    float64
3   Weight       1000 non-null    float64
4   Gender       1000 non-null    object
5   Department   1000 non-null    object
6   GPA          1000 non-null    float64
7   Semester     1000 non-null    int64
8   Hosteller    1000 non-null    object
9   Grade        1000 non-null    object
dtypes: float64(2), int64(4), object(4)

```

3. Store the data in a Python dictionary named students.

```

1 students = student_data.to_dict(orient='records')
2
3 print("DataFrame converted to a list of dictionaries named 'students'.")
4 print("Displaying the first 5 entries of the 'students' dictionary:")
5 display(students[:5])
6
7 print(f"\nTotal number of student records: {len(students)}")

```

Output

DataFrame converted to a list of dictionaries named 'students'.
Displaying the first 5 entries of the 'students' dictionary:

```

[{'ID': 1,
  'Age': 19,

```

```

    'Height': 151.19,
    'Weight': 70.54,
    'Gender': 'Female',
    'Department': 'IT',
    'CGPA': 8.81,
    'Semester': 6,
    'Hosteller': 'Yes',
    'Grade': 'B'},
{'ID': 2,
 'Age': 23,
 'Height': 150.46,
 'Weight': 80.77,
 'Gender': 'Female',
 'Department': 'EC',
 'CGPA': 6.43,
 'Semester': 8,
 'Hosteller': 'No',
 'Grade': 'D'},
{'ID': 3,
 'Age': 22,
 'Height': 163.27,
 'Weight': 58.03,
 'Gender': 'Female',
 'Department': 'EE',
 'CGPA': 8.67,
 'Semester': 8,
 'Hosteller': 'Yes',
 'Grade': 'B'},
{'ID': 4,
 'Age': 25,
 'Height': 152.46,
 'Weight': 70.33,
 'Gender': 'Male',
 'Department': 'CE',
 'CGPA': 8.86,
 'Semester': 1,
 'Hosteller': 'Yes',
 'Grade': 'B'},
{'ID': 5,
 'Age': 18,
 'Height': 173.93,
 'Weight': 85.3,
 'Gender': 'Female',
 'Department': 'IT',
 'CGPA': 9.13,

```

```

'Semester': 6,
'Hosteller': 'No',
'Grade': 'A']
Total number of student records: 1000

```

4. Create a Pandas DataFrame named df from the dictionary.

```

1 df = pd.DataFrame(students)
2
3 print("DataFrame 'df' created from 'students' list of dictionaries.")
4 print("Displaying the first 5 rows of df:")
5 display(df.head())
6
7 print(f"\nDataset shape: {df.shape}")
8 print("Dataset info:")
9 display(df.info())

```

Output

```

-- DataFrame 'df' created from 'students' list of dictionaries.
Displaying the first 5 rows of df:

```

ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller	Grade
0	1	18	151.13	Female	IT	8.91	6	Yes	B
1	2	23	150.48	Female	EC	8.43	6	No	D
2	3	22	163.27	Female	EE	8.57	6	Yes	B
3	4	25	152.48	Male	CE	8.95	1	Yes	B
4	5	18	173.83	Female	IT	8.13	6	No	A

5. Display the first five rows and last five rows of the DataFrame.

```

1 print("\n--- First 5 rows of the DataFrame (df.head()) ---")
2 display(df.head())
3
4 print("\n--- Last 5 rows of the DataFrame (df.tail()) ---")
5 display(df.tail())

```

```

-- First 5 rows of the DataFrame (df.head()) --

```

ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller	Grade
0	1	18	151.13	Female	IT	8.91	6	Yes	B
1	2	23	150.48	Female	EC	8.43	6	No	D
2	3	22	163.27	Female	EE	8.57	6	Yes	B
3	4	25	152.48	Male	CE	8.95	1	Yes	B
4	5	18	173.83	Female	IT	8.13	6	No	A

```

-- Last 5 rows of the DataFrame (df.tail()) --

```

ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller	Grade
995	995	16	174.57	Male	ME	8.55	7	No	A
996	997	19	162.08	Male	CS	8.91	4	Yes	D
997	998	21	167.82	Female	CE	8.98	4	Yes	A
998	999	21	169.72	Male	IT	8.43	5	Yes	D
999	1000	25	175.82	Female	CE	7.55	6	Yes	D

Output

Part B: Exploring the DataFrame

1. Display the first seven rows.

```
1 print("\n--- First 7 rows of the DataFrame ---")
2 display(df.head(7))
```

Output

--- First 7 rows of the DataFrame ---

ID	Age	Height	Weight	Gender	Department	GPA	Semester	Hosteller	Grade
1	18	151.19	70.94	Female	IT	8.81	5	Yes	B
2	23	150.46	50.77	Female	EC	6.43	5	No	D
3	22	163.27	56.03	Female	EE	8.67	5	Yes	B
4	25	152.46	70.33	Male	CE	8.86	1	Yes	B
5	16	173.03	55.30	Female	IT	9.13	5	No	A
6	19	165.44	60.65	Female	EC	8.57	2	Yes	B
7	21	166.49	58.16	Male	CS	8.27	1	Yes	B

2. Display the last three rows.

```
1 print("\n--- Last 3 rows of the DataFrame ---")
2 display(df.tail(3))
```

Output

--- Last 3 rows of the DataFrame ---

ID	Age	Height	Weight	Gender	Department	GPA	Semester	Hosteller	Grade
997	21	161.82	73.22	Female	CE	9.09	4	Yes	A
998	21	169.72	74.11	Male	IT	6.43	5	Yes	D
999	25	179.52	92.49	Female	CE	7.89	5	Yes	C

3. Display the shape of the DataFrame.

```
1 print("\n--- Shape of the DataFrame ---")
2 print(df.shape)
```

Output

```
--- Shape of the DataFrame ---
(1000, 10)
```

4. Display the names of all columns

```
1 print("\n--- Column Names of the DataFrame ---")
2 print(df.columns.tolist())
```

Output

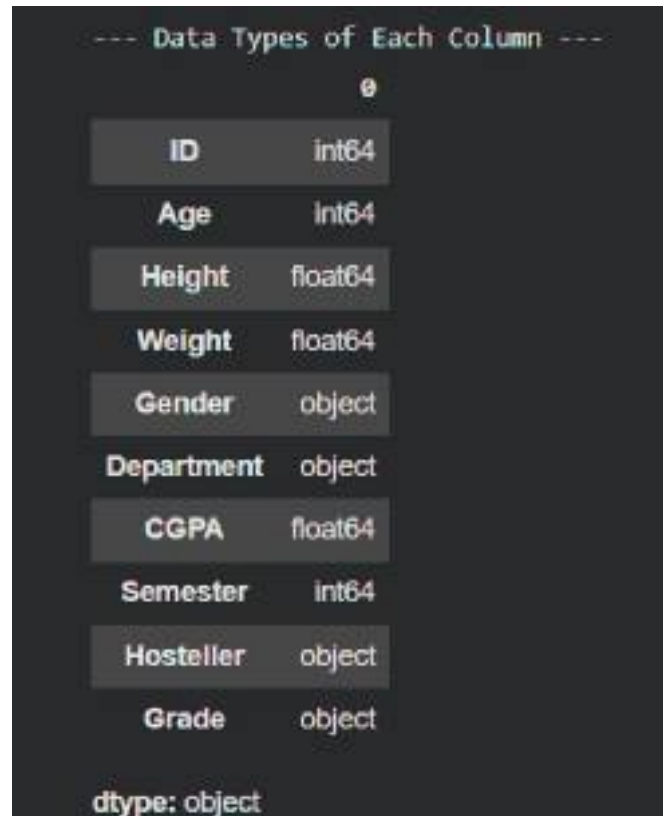
```
--- Column Names of the DataFrame ---
```

```
['ID', 'Age', 'Height', 'Weight', 'Gender', 'Department', 'CGPA', 'Semester', 'Hosteller', 'Grade']
```

5. Display the data type of each column.

```
1 print("\n--- Data Types of Each Column ---")
2 display(df.dtypes)
```

Output



```
--- Data Types of Each Column ---
0
ID      int64
Age     int64
Height  float64
Weight  float64
Gender  object
Department object
CGPA     float64
Semester int64
Hosteller object
Grade   object
dtype: object
```

6. Display summary information using info().

```
1 print("\n--- Summary Information of the DataFrame ---")
2 df.info()
```

Output

```
--- Summary Information of the DataFrame ---
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
```

```
Data columns (total 10 columns):
```

#	Column	Non-Null Count	Dtype
0	ID	1000 non-null	int64

```

1  Age          1000 non-null  int64
2  Height       1000 non-null  float64
3  Weight       1000 non-null  float64
4  Gender       1000 non-null  object
5  Department   1000 non-null  object
6  CGPA         1000 non-null  float64
7  Semester     1000 non-null  int64
8  Hosteller    1000 non-null  object
9  Grade        1000 non-null  object
dtypes: float64(3), int64(3), object(4)
memory usage: 78.3+ KB

```

7. Display descriptive statistics of all numerical columns.

```

1  print("\n--- Descriptive Statistics for Numerical Columns ---")
2  display(df.describe())

```

Output

```

--- Descriptive statistics for numerical columns ---

```

	ID	Age	Height	Weight	CGPA	Semester
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	500.500000	21.366000	169.944530	66.609180	7.663160	4.622000
std	269.619436	2.365662	11.755481	11.663919	1.114366	2.299833
min	1.000000	18.000000	160.010000	50.100000	6.000000	1.000000
25%	250.750000	19.000000	168.677500	59.182500	7.062600	3.000000
50%	500.500000	21.000000	169.375000	60.665000	8.000000	5.000000
75%	750.250000	24.000000	180.445000	70.705000	8.880000	7.000000
max	1000.000000	25.000000	181.950000	80.000000	10.000000	8.000000

Part C: Reading Data

1. Display the Age column.

```
1 print("\n--- Displaying the 'Age' column ---")
2 display(df['Age'])
```

Output



The screenshot shows the output of the code in a Jupyter Notebook. It displays the 'Age' column of a DataFrame with 1000 rows. The output is formatted as a table with two columns: an index and the 'Age' values. The first five rows are visible, followed by an ellipsis, and then the last five rows (995 to 999). Below the table, it indicates '1000 rows x 1 columns' and 'dtype: int64'.

	Age
0	19
1	23
2	22
3	25
4	18
...	...
995	18
996	19
997	21
998	21
999	25

1000 rows x 1 columns
dtype: int64

2. Display the Age and CGPA columns.

```
1 print("\n--- Displaying 'Age' and 'CGPA' columns ---")
2 display(df[['Age', 'CGPA']])
```

Output

--- Displaying 'Age' and 'CGPA' columns ---

	Age	CGPA
0	19	8.81
1	23	6.43
2	22	8.67
3	25	8.86
4	18	9.13
...	...	...
995	18	9.89
996	19	6.61
997	21	9.69
998	21	6.43
999	25	7.59

1000 rows x 2 columns

3. Display rows 2 to 6.

```
1 print("\n--- Displaying Rows 2 to 6 (inclusive) ---")
2 display(df.iloc[2:7])
```

Output

```

--- Displaying Rows 2 to 6 (Inclusive) ---

```

ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hostaller	Grade
2	3	22	183.27	94.00	Female	EE	8.97	3	Yes
3	4	25	182.48	70.55	Male	CE	8.86	1	Yes
4	5	18	173.93	85.30	Female	IT	9.13	6	No
5	6	19	185.44	69.66	Female	EC	8.67	2	Yes
6	7	21	199.49	85.16	Male	CS	8.27	1	Yes

4. Display rows 2 to 6 containing only the Age, Department and CGPA columns.

```
1 print("\n--- Displaying Rows 2 to 6 with 'Age', 'Department', and 'CGPA' ←
    columns ---")
2 display(df.iloc[2:7][['Age', 'Department', 'CGPA']])
```

Output

```

--- Displaying rows 2 to 6 with 'Age', 'Department', and 'CGPA' columns ---

```

	Age	Department	CGPA
2	22	EE	8.57
3	25	CE	8.56
4	18	IT	8.13
5	19	EC	8.57
6	21	CS	8.27

5. Display rows 1, 5, 8 and 10 containing only Height and Weight.

```

1 print("\n--- Displaying rows 1, 5, 8, and 10 with 'Height' and 'Weight' ←
    columns ---")
2 display(df.iloc[[0, 4, 7, 9]][['Height', 'Weight']])

```

Output

```

--- Displaying rows 2 to 6 with 'Age', 'Department', and 'CGPA' columns ---

```

	Age	Department	CGPA
2	22	EE	8.57
3	25	CE	8.56
4	18	IT	8.13
5	19	EC	8.57
6	21	CS	8.27

6. Display the Name, Department and CGPA of students having CGPA greater than 8

```

1 print("\n--- Students with CGPA greater than 8 (ID, Department, CGPA) ---←
    ")
2 display(df[df['CGPA'] > 8][['ID', 'Department', 'CGPA']])

```

Output

```

--- Displaying rows 1, 5, 8, and 10 with 'Height' and 'Weight' columns ---

```

	Height	Weight
0	151.19	70.54
4	173.50	85.30
7	151.53	75.75
9	155.28	68.50

7. Display the student whose ID is 5.

```

1 print("\n--- Student with ID 5 ---")
2 display(df[df['ID'] == 5])

```

Output

--- Students with CGPA greater than 8 (ID, Department, CGPA) ---

ID	Department	CGPA
0	1	IT 8.81
2	3	EE 8.67
3	4	CE 8.86
4	5	IT 9.13
5	6	EC 8.57
...	...	...
989	890	CS 8.73
992	893	EE 8.89
993	894	EE 8.35
995	996	ME 9.89
997	998	CE 9.69

498 rows x 3 columns

8. Display all students belonging to the CSE department.

```
1 print("\n--- Students from CS Department ---")
2 display(df[df['Department'] == 'CS'])
```

Output

Student with ID 5

ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller	Grade
4	5	16	175.00	85.3	Female	IT 9.13	8	No	A

9. Display students having CGPA greater than 9

```
1 print("\n--- Students with CGPA greater than 9 ---")
2 display(df[df['CGPA'] > 9])
```

Output

--- students from CS department ---

ID	Age	Height	Weight	Gender	Department	GPA	Semester	Hosteller	Grade	
8	7	21	106.48	66.18	Male	CS	8.27	1	Yes	B
9	9	19	153.29	52.34	Male	CS	8.43	3	No	D
21	22	21	100.66	82.73	Female	CS	8.73	4	Yes	B
36	37	21	175.29	68.38	Female	CS	8.42	1	Yes	D
38	40	23	108.88	117.77	Male	CS	7.43	6	Yes	C
...	...	...	...	...	...	...	...	...	...	
976	977	25	160.62	56.50	Female	CS	8.53	7	No	D
977	978	21	161.43	71.78	Male	CS	8.54	8	No	B
988	987	24	155.44	68.73	Female	CS	9.48	1	Yes	A
989	990	24	174.01	77.38	Female	CS	8.73	4	No	B
995	997	19	102.06	64.19	Male	CS	9.81	4	Yes	D

165 rows x 10 columns

10. Display the first three rows using integer indexing.

```
1 print("\n--- First three rows using integer indexing (iloc[0:3]) ---")
2 display(df.iloc[0:3])
```

Output

Students with GPA greater than 8										
ID	Age	Height	Weight	Gender	Department	GPA	Semester	Hosteller	Grade	
4	5	18	173.30	85.30	Female	IT	9.15	6	No	A
7	6	22	191.53	75.75	Female	ME	9.87	3	No	A
13	14	20	184.49	69.42	Male	EC	9.06	6	Yes	A
16	17	18	181.49	67.43	Male	IT	9.00	7	No	A
18	19	18	174.88	88.31	Female	IT	9.59	7	Yes	A
...										
988	987	24	155.44	68.73	Female	CS	9.48	1	Yes	A
988	989	19	167.82	66.34	Female	ME	9.04	3	Yes	A
993	994	24	161.50	64.40	Male	EE	9.35	3	No	A
995	996	18	174.97	66.82	Male	ME	9.89	7	No	A
997	998	21	181.82	75.22	Female	CE	9.88	4	Yes	A

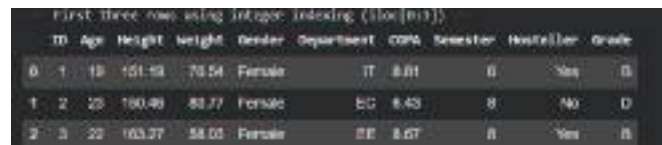
219 rows x 10 columns

Part D: Creating New Data

1. Add the student to the end of the DataFrame.

```
1 newStudent = {
2     'ID': 21,
3     'Age': 20,
4     'Height': 172.5,
5     'Weight': 68.4,
6     'Gender': 'Male',
7     'Department': 'CSE',
8     'CGPA': 8.75,
9     'Semester': 5,
10    'Hosteller': 'Yes',
11    'Grade': 'A'
12 }
13
14 # Convert the new student dictionary to a DataFrame row
15 new_student_df = pd.DataFrame([newStudent])
16
17 # Append the new student to the existing DataFrame
18 df = pd.concat([df, new_student_df], ignore_index=True)
19
20 print("\n--- New student added to DataFrame. Displaying the last 5 rows ←
      of the updated DataFrame ---")
21 display(df.tail())
```

Output



	ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller	Grade
0	1	19	161.59	70.54	Female	IT	8.81	6	Yes	B
1	2	20	160.46	80.77	Female	EC	8.43	8	No	D
2	3	22	163.37	58.03	Female	EE	8.67	8	Yes	B

2. Create a new column named BMI by computing the Body Mass Index of each student using

$$\text{BMI} = \frac{\text{Weight (kg)}}{\left(\frac{\text{Height (cm)}}{100}\right)^2}$$

```
1 # Create a new column named BMI
2 df['BMI'] = df['Weight'] / ((df['Height'] / 100)**2)
3
4 print("\n--- DataFrame with new 'BMI' column (first 5 rows) ---")
5 display(df.head())
```

Output

new student added to database, displaying the last 5 rows of the updated database

ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller	Grade	
998	997	19	162.88	84.15	Male	CE	6.89	6	Yes	D
997	996	21	163.82	75.22	Female	CE	8.89	4	Yes	A
996	995	21	169.72	74.11	Male	IT	6.43	5	Yes	D
995	1000	25	175.62	52.40	Female	CE	7.89	6	Yes	C
1000	21	25	173.50	68.40	Male	CE	8.75	5	Yes	A

3. Create a Boolean column named Passed.

```

1 df['Passed'] = df['CGPA'] >= 6.0
2
3 print("\n--- DataFrame with new 'Passed' column (first 5 rows) ---")
4 display(df.head())

```

Output

Database with new 'Passed' column (first 5 rows)										
ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller	Grade	Passed
1	19	161.19	73.54	Female	IT	8.81	6	Yes	B	38.890531
2	23	160.48	80.77	Female	EC	6.45	6	No	D	38.876613
3	22	165.27	88.03	Female	EE	8.87	6	Yes	B	21.788888
4	25	162.48	73.33	Male	CE	8.86	1	Yes	B	38.257204
5	18	175.00	85.30	Female	IT	9.13	6	No	A	28.196814

4. Create a BonusCGPA column by adding 0.25 to every CGPA.

```

1 df['BonusCGPA'] = df['CGPA'] + 0.25
2
3 print("\n--- DataFrame with new 'BonusCGPA' column (first 5 rows) ---")
4 display(df.head())

```

Output

database with new 'Passed' column (first 5 rows)

ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller	Grade	BPC	Passed
1	19	161.19	73.54	Female	IT	8.81	6	Yes	B	38.890531	True
2	23	160.48	80.77	Female	EC	6.45	6	No	D	38.876613	False
3	22	165.27	88.03	Female	EE	8.87	6	Yes	B	21.788888	True
4	25	162.48	73.33	Male	CE	8.86	1	Yes	B	38.257204	True
5	18	175.00	85.30	Female	IT	9.13	6	No	A	28.196814	True

5. Create a categorical column named Performance according to the following criteria.
 - Excellent if CGPA >= 9
 - Good if 8 < CGPA < 9
 - Average, Poor if CGPA < 7

```

1 conditions = [
2     df['CGPA'] >= 9,

```

```

3      (df['CGPA'] >= 8) & (df['CGPA'] < 9),
4      (df['CGPA'] >= 7) & (df['CGPA'] < 8),
5      df['CGPA'] < 7
6  ]
7
8  choices = ['Excellent', 'Good', 'Average', 'Poor']
9
10 df['Performance'] = np.select(conditions, choices, default='Unknown')
11
12 print("\n--- DataFrame with new 'Performance' column (first 5 rows) ---")
13 display(df.head())

```

Output

DataFrame with new 'Performance' column (first 5 rows)

ID	Age	Height	Weight	Gender	Department	CGPA	Exercise	Hosteller	Grade	MTI	Passed	SchoolCGPA
1	19	161.12	75.35	Female	IT	8.81	8	Yes	B	33.830031	True	9.36
2	20	156.45	85.17	Female	CO	6.43	8	No	D	35.673014	True	6.28
3	22	161.27	80.00	Female	EC	8.67	8	Yes	B	25.700089	True	6.30
4	25	152.45	75.35	Male	CE	8.86	1	Yes	B	30.257284	True	8.71
5	18	173.00	85.30	Female	IT	9.15	8	No	A	26.900014	True	9.36

6. Introduce missing values into the DataFrame by setting the following entries to NaN.
 - (a) Set the CGPA of the student whose ID is 5 to NaN.
 - (b) Set the Height of the student whose ID is 12 to NaN.
 - (c) Set the Department of the student whose ID is 18 to NaN. (Hint: Use np.nan.)
-

```

1  # (a) Set the CGPA of the student whose ID is 5 to NaN.
2  df.loc[df['ID'] == 5, 'CGPA'] = np.nan
3
4  # (b) Set the Height of the student whose ID is 12 to NaN.
5  df.loc[df['ID'] == 12, 'Height'] = np.nan
6
7  # (c) Set the Department of the student whose ID is 18 to NaN.
8  df.loc[df['ID'] == 18, 'Department'] = np.nan
9
10 print("--- DataFrame entries after introducing NaNs ---")
11 print("Student with ID 5 (CGPA should be NaN):")
12 display(df[df['ID'] == 5])
13
14 print("\nStudent with ID 12 (Height should be NaN):")
15 display(df[df['ID'] == 12])
16
17 print("\nStudent with ID 18 (Department should be NaN):")
18 display(df[df['ID'] == 18])

```

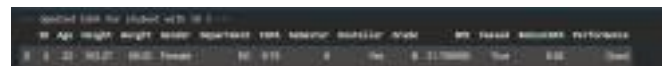
Output

Part E: Updating Data

1. Update the CGPA of the student whose ID is 3 to 9.75.

```
1 # Update the CGPA of the student whose ID is 3 to 9.75
2 df.loc[df['ID'] == 3, 'CGPA'] = 9.75
3
4 print("--- Updated CGPA for student with ID 3 ---")
5 display(df[df['ID'] == 3])
```

Output



ID	Age	Height	Weight	Gender	Department	CGPA	Minor	Module	Score	Mid	Final	Average	Performance
3	22	163.27	59.02	Female	EC	9.75	0	No	0	21.75000	True	9.75	Good

2. Change the Department of student ID 8 to CSE.

```
1 df.loc[df['ID'] == 8, 'Department'] = 'CSE'
2
3 print("--- Updated Department for student with ID 8 ---")
4 display(df[df['ID'] == 8])
```

Output

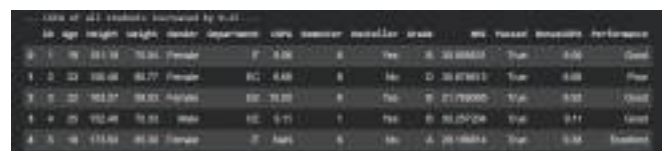


ID	Age	Height	Weight	Gender	Department	CGPA	Minor	Module	Score	Mid	Final	Average	Performance
8	22	172.25	75.92	Female	CSE	9.75	0	No	0	22.50000	True	10.75	Excellent

3. Increase the CGPA of every student by 0.25.

```
1 df['CGPA'] += 0.25
2
3 print("--- CGPA of all students increased by 0.25 ---")
4 display(df.head())
```

Output



ID	Age	Height	Weight	Gender	Department	CGPA	Minor	Module	Score	Mid	Final	Average	Performance
0	19	161.18	70.91	Female	EC	9.00	0	Yes	0	20.00000	True	9.00	Good
1	22	168.08	62.77	Female	EC	9.00	0	No	0	20.00000	True	9.00	Good
2	22	163.27	59.02	Female	EC	9.00	0	No	0	21.75000	True	9.00	Good
3	22	163.27	59.02	Female	EC	9.75	0	No	0	21.75000	True	9.75	Good
4	18	171.52	69.92	Female	EC	9.00	0	No	0	20.00000	True	9.00	Good

4. Increase the Weight of every student by 2 kg.

```

1 df['Weight'] += 2
2
3 print("--- Weight of all students increased by 2 kg ---")
4 display(df.head())

```

Output

Weight of all students increased by 2 kg													
ID	Age	Height	Weight	Gender	Department	GPA	IsTeacher	NotesCmt	Grade	MCQ_Passed	NotesGPA	Performance	
1	1	16	51.16	Female	IT	8.88	0	Yes	B	35.918121	True	8.88	Good
1	2	22	100.48	Male	IT	8.88	0	No	D	30.418173	True	8.88	Poor
2	5	20	100.07	Male	IT	8.88	0	Yes	B	31.703000	True	8.88	Good
2	6	26	100.08	Male	IT	8.88	1	Yes	B	35.327583	True	8.88	Good
3	5	18	173.82	Female	IT	8.88	0	No	A	38.788814	True	8.88	Excellent

5. Change all Grade C students to Grade B.

```

1 print("--- Grade distribution BEFORE change ---")
2 display(df['Grade'].value_counts().sort_index())
3
4 df.loc[df['Grade'] == 'C', 'Grade'] = 'B'
5
6 print("\n--- All 'Grade C' students have been changed to 'Grade B'. ---")
7 print("--- Grade distribution AFTER change ---")
8 display(df['Grade'].value_counts().sort_index())
9
10 print("\n--- First 10 rows of the updated DataFrame: ---")
11 display(df.head(10))

```

Output

--- Grade distribution BEFORE change ---

Grade	count
A	221
B	281
C	263
D	236

dtype: int64

--- All 'Grade C' students have been changed to 'Grade B'. ---

--- Grade distribution AFTER change ---

Grade	count
A	221
B	544
D	236

dtype: int64

First 10 rows of the updated DataFrame:															
	ID	Age	Height	Weight	Gender	Height/Weight	DOB	Transfer	Transfer	Transfer	Scale	MR	Transfer	Transfer	Performance
0	1	18	180.19	72.54	Female	9	0.00	0	Yes	0	30.00000	Yes	0.00	Good	
1	2	20	180.48	80.70	Female	10	0.00	0	Yes	0	30.00000	Yes	0.00	Good	
2	3	20	180.27	80.00	Female	10	0.00	0	Yes	0	30.00000	Yes	0.00	Good	
3	4	20	180.48	70.33	Male	10	0.00	1	Yes	0	30.00000	Yes	0.00	Good	
4	5	18	180.00	80.00	Female	9	0.00	0	Yes	0	30.00000	Yes	0.00	Good	
5	6	18	180.00	70.00	Female	9	0.00	0	Yes	0	30.00000	Yes	0.00	Good	
6	7	20	180.00	80.00	Female	10	0.00	0	Yes	0	30.00000	Yes	0.00	Good	
7	8	20	180.00	70.00	Female	10	0.00	0	Yes	0	30.00000	Yes	0.00	Good	
8	9	18	180.00	80.00	Female	9	0.00	0	Yes	0	30.00000	Yes	0.00	Good	
9	10	20	180.00	80.00	Female	10	0.00	0	Yes	0	30.00000	Yes	0.00	Good	

6. Display the updated DataFrame.

```
1 print("--- Fully Updated DataFrame ---")
2 display(df)
```

Output

1111 entries of the updated DataFrame:														
	ID	Age	Height	Weight	Gender	Height/Weight	DOB	Transfer	Transfer	Transfer	Scale	MR	Transfer	Performance
0	1	18	180.19	72.54	Female	9	0.00	0	Yes	0	30.00000	Yes	0.00	Good
1	2	20	180.48	80.70	Female	10	0.00	0	Yes	0	30.00000	Yes	0.00	Good
2	3	20	180.27	80.00	Female	10	0.00	0	Yes	0	30.00000	Yes	0.00	Good
3	4	20	180.48	70.33	Male	10	0.00	1	Yes	0	30.00000	Yes	0.00	Good
4	5	18	180.00	80.00	Female	9	0.00	0	Yes	0	30.00000	Yes	0.00	Good
5	6	18	180.00	70.00	Female	9	0.00	0	Yes	0	30.00000	Yes	0.00	Good
6	7	20	180.00	80.00	Female	10	0.00	0	Yes	0	30.00000	Yes	0.00	Good
7	8	20	180.00	70.00	Female	10	0.00	0	Yes	0	30.00000	Yes	0.00	Good
8	9	18	180.00	80.00	Female	9	0.00	0	Yes	0	30.00000	Yes	0.00	Good
9	10	20	180.00	80.00	Female	10	0.00	0	Yes	0	30.00000	Yes	0.00	Good

Part F: Deleting Data

1. Delete the student whose ID is 10.

```
1 print("\n--- Deleting student with ID 10 ---")
2 df = df[df['ID'] != 10]
3
4 print("Student with ID 10 deleted. Displaying the first 5 rows of the ←
      updated DataFrame:")
5 display(df.head())
```

Output

Deleting student with ID 10 ---													
Student with ID 10 deleted. Displaying the first 5 rows of the updated DataFrame:													
ID	age	height	weight	gender	department	class	semester	enrollment	grade	gpa	passed	attendance	performance
1	18	180.00	72.00	Female	IT	001	0	No	30.000000	True	0.00	Good	
2	20	180.00	72.00	Female	SC	001	0	No	30.000000	True	0.00	Good	
3	22	180.00	80.00	Female	SC	1000	0	No	31.000000	True	0.00	Good	
4	20	180.00	72.00	Male	CS	001	1	No	30.000000	True	0.00	Good	
5	18	170.00	67.50	Female	IT	001	0	No	30.000000	True	0.00	Excellent	

2. Delete the Grade column.

```
1 print("\n--- Deleting the 'Grade' column ---")
2 df = df.drop(columns=['Grade'])
3
4 print("The 'Grade' column has been deleted. Displaying the first 5 rows ←
      of the updated DataFrame:")
5 display(df.head())
```

Output



Deleting the 'Grade' column ---
The 'Grade' column has been deleted. Displaying the first 5 rows of the updated DataFrame:

ID	age	height	weight	gender	department	class	semester	enrollment	gpa	passed	attendance	performance
1	18	180.00	72.00	Female	IT	001	0	No	30.000000	True	0.00	Good
2	20	180.00	72.00	Female	SC	001	0	No	30.000000	True	0.00	Good
3	22	180.00	80.00	Female	SC	1000	0	No	31.000000	True	0.00	Good
4	20	180.00	72.00	Male	CS	001	1	No	30.000000	True	0.00	Good
5	18	170.00	67.50	Female	IT	001	0	No	30.000000	True	0.00	Excellent

3. Delete the first row.

```
1 print("\n--- Deleting the first row ---")
2 df = df.drop(index=df.index[0])
3
4 print("The first row has been deleted. Displaying the first 5 rows of the ←
      updated DataFrame:")
5 display(df.head())
```

Output

Deleting the 'Grade' column
The 'Grade' column has been deleted. Displaying the first 5 rows of the updated dataframe:

ID	Age	Height	Weight	Gender	Department	GPA	Semester	Reseller	WFL	Passed	ScoreGPA	Performance
1	1	18	161.15	Female	IT	8.88	6	No	30.838821	True	8.90	Good
1	1	18	161.45	Female	EC	8.88	6	No	30.878815	True	8.88	Good
2	2	22	161.21	Female	EE	10.00	6	No	31.750000	True	9.80	Good
2	4	22	161.45	Male	CE	8.11	1	No	30.217004	True	8.11	Good
4	2	18	171.80	Female	IT	8.88	6	No	30.300014	True	8.88	Excellent

4. Delete all students having CGPA less than 7.

```

1 print("\n--- Deleting students with CGPA less than 7 ---")
2 df = df[df['CGPA'] >= 7]
3
4 print("Students with CGPA less than 7 deleted. Displaying the first 5 ←
      rows of the updated DataFrame:")
5 display(df.head())

```

Output

Deleting the first row
The first row has been deleted. Displaying the first 5 rows of the updated dataframe:

ID	Age	Height	Weight	Gender	Department	GPA	Semester	Reseller	WFL	Passed	ScoreGPA	Performance
1	2	22	160.48	Female	EC	8.88	6	No	30.878815	True	8.88	Good
2	2	22	160.21	Female	EE	10.00	6	No	31.750000	True	9.80	Good
3	2	20	162.46	Male	CE	8.11	1	No	30.217004	True	8.11	Good
4	2	18	171.80	Female	IT	8.88	6	No	30.300014	True	8.88	Excellent
5	2	18	160.44	Female	EC	8.88	6	No	30.878815	True	8.88	Good

5. Reset the DataFrame index.

```

1 print("\n--- Resetting the DataFrame index ---")
2 df = df.reset_index(drop=True)
3
4 print("DataFrame index reset. Displaying the first 5 rows of the updated ←
      DataFrame:")
5 display(df.head())

```

Output

Deleting students with CGPA less than 7
Students with CGPA less than 7 deleted. Displaying the first 5 rows of the updated dataframe:

ID	Age	Height	Weight	Gender	Department	GPA	Semester	Reseller	WFL	Passed	ScoreGPA	Performance
2	2	22	160.21	Female	EE	10.00	6	No	31.750000	True	9.80	Good
3	2	20	162.46	Male	CE	8.11	1	No	30.217004	True	8.11	Good
4	2	18	171.80	Female	IT	8.88	6	No	30.300014	True	8.88	Excellent
5	1	21	160.46	Male	CE	8.88	1	No	31.204991	True	8.88	Good
7	2	22	161.02	Female	CE	10.10	6	No	33.880004	True	10.10	Excellent

6. Display the updated DataFrame.

```
1 print("\n--- Displaying the fully updated DataFrame ---")
2 display(df)
```

Output

Scrolling the DataFrame Index

56/67 rows (rows reset). Displaying the first 5 rows of the updated DataFrame.

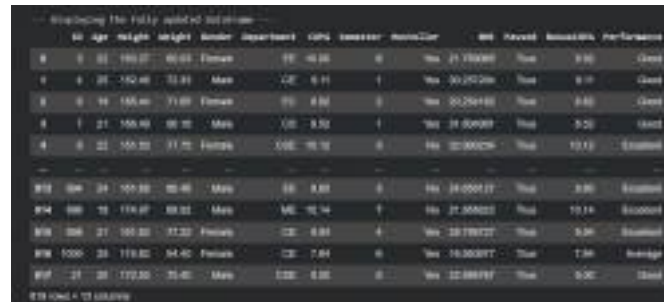
	id	age	height	weight	gender	department	salary	tenure	reseller	reseller_id	WFL	WFLpass	Responsible	Performance
0	A	30	160.27	60.03	Female	FE	6.66	6	Yes	21749660	True	6.66	Good	
1	A	26	152.65	72.55	Male	CE	6.11	1	Yes	16267564	True	6.11	Good	
2	A	31	156.66	71.69	Female	FE	6.61	2	Yes	20264762	True	6.61	Good	
3	T	21	156.69	66.16	Male	CE	6.62	1	Yes	21164691	True	6.62	Good	
4	A	31	151.25	71.76	Female	FE	6.62	6	Yes	20886236	True	6.62	Excellent	

Part G: Filtering and Sorting

1. Display students whose Age is greater than 20.

```
1 print("\n--- Students with Age greater than 20 ---")
2 display(df[df['Age'] > 20])
```

Output



Displaying the fully edited notebook

	ID	Age	Height	Weight	Gender	Department	CGPA	Hosteller	HostelID	SRG	Passed	ResultCGPA	Performance
0	0	22	150.27	55.03	Female	TE	6.56	0	Yes	21.70000	True	6.56	Good
1	0	20	152.45	73.81	Male	CSE	6.11	1	Yes	30.20000	True	6.11	Good
2	0	19	155.40	71.89	Female	TE	6.80	0	Yes	30.20000	True	6.80	Good
3	1	21	155.49	66.70	Male	CSE	8.52	1	Yes	31.00000	True	8.52	Good
4	0	22	155.00	71.70	Female	CSE	7.92	0	Yes	32.00000	True	7.92	Good
...													
873	004	24	155.30	60.46	Male	SE	8.89	0	Yes	34.00000	True	8.89	Excellent
874	000	19	174.67	89.02	Male	ME	10.74	1	Yes	37.50000	True	10.74	Excellent
875	000	21	160.00	72.00	Female	CSE	6.80	0	Yes	33.70000	True	6.80	Good
876	1000	20	170.00	84.40	Female	CSE	7.84	0	Yes	35.00000	True	7.84	Good
877	21	20	170.00	70.40	Male	CSE	6.80	0	Yes	32.00000	True	6.80	Good

878 rows x 14 columns

2. Display Female students belonging to the CSE department.

```
1 print("\n--- Female students belonging to the CSE department ---")
2 display(df[(df['Gender'] == 'Female') & (df['Department'] == 'CSE')])
```

Output



Displaying the fully edited notebook

	ID	Age	Height	Weight	Gender	Department	CGPA	Hosteller	HostelID	SRG	Passed	ResultCGPA	Performance
0	0	22	150.27	55.03	Female	TE	6.56	0	Yes	21.70000	True	6.56	Good
1	0	20	152.45	73.81	Male	SE	6.11	1	Yes	30.20000	True	6.11	Good
2	0	19	155.40	71.89	Female	TE	6.80	0	Yes	30.20000	True	6.80	Good
3	0	22	155.00	71.70	Female	CSE	7.92	0	Yes	32.00000	True	7.92	Good
4	0	22	155.00	71.70	Female	CSE	7.92	0	Yes	32.00000	True	7.92	Good
...													
873	004	24	155.30	60.46	Male	SE	8.89	0	Yes	34.00000	True	8.89	Excellent
874	000	19	174.67	89.02	Male	ME	10.74	1	Yes	37.50000	True	10.74	Excellent
875	000	21	160.00	72.00	Female	CSE	6.80	0	Yes	33.70000	True	6.80	Good
876	1000	20	170.00	84.40	Female	CSE	7.84	0	Yes	35.00000	True	7.84	Good

878 rows x 14 columns

3. Display Hostellers having CGPA greater than 8.

```
1 print("\n--- Hostellers with CGPA greater than 8 ---")
2 display(df[(df['Hosteller'] == 'Yes') & (df['CGPA'] > 8)])
```

Output

Hostellers belonging to the CG department													
	ID	Age	Height	Weight	Department	CGPA	Hosteller	HostelID	SRG	Passed	ResultCGPA	Performance	
0	0	22	150.27	55.03	Female	CSE	7.92	0	Yes	32.00000	True	7.92	Good

- Sort the DataFrame by CGPA in ascending order.

```
1 print("\n--- DataFrame sorted by CGPA in ascending order ---")
2 display(df.sort_values(by='CGPA', ascending=True))
```

Output

id	age	height	weight	gender	department	CGPA	semester	hostelCar	fees Paid	attendance	performance
9	2	150.25	60.69	Female	EE	8.08	6	No	2178000	True	Good
1	4	150.49	72.53	Male	CE	8.11	1	No	16251204	True	Good
3	8	150.44	71.63	Female	EE	8.62	3	No	3429492	True	Good
5	7	150.49	60.48	Male	CE	8.52	1	No	11894887	True	Good
7	10	150.55	62.35	Male	CE	8.54	6	No	35388718	True	Good
...	...	...	...	...	...	...	...	...	...	...	...
998	888	151	150.35	Male	ET	8.94	7	No	21428403	True	Good
999	888	151	157.52	Female	ME	8.88	3	No	36333333	True	Excellent
1000	888	151	150.55	Female	CE	8.74	6	No	31831987	True	Good
1001	888	151	154.25	Female	CE	8.94	4	No	28796727	True	Excellent
1002	27	151	172.55	Male	CE	8.58	6	No	33388797	True	Good

- Sort the DataFrame by Height in descending order.

```
1 print("\n--- DataFrame sorted by Height in descending order ---")
2 display(df.sort_values(by='Height', ascending=False))
```

Output

id	age	height	weight	gender	department	CGPA	semester	hostelCar	fees Paid	attendance	performance
998	888	151	150.35	Male	ET	8.94	7	No	21428403	True	Good
999	888	151	157.52	Female	CE	8.88	3	No	36333333	True	Good
1000	888	151	150.55	Female	CE	8.74	6	No	31831987	True	Good
1001	888	151	154.25	Female	CE	8.94	4	No	28796727	True	Good
1002	27	151	172.55	Male	CE	8.58	6	No	33388797	True	Good
...	...	...	...	...	...	...	...	...	...	...	...
995	888	149	148.57	Male	EE	7.98	1	No	22388273	True	Good
996	888	149	145.52	Female	CE	7.98	6	No	15404736	True	Good
997	888	149	148.83	Female	EE	7.98	1	No	25571258	True	Good
998	888	149	150.44	Male	ME	7.98	7	No	22388273	True	Good
999	888	149	143.83	Female	ME	7.98	6	No	20299999	True	Good

- Display the top five students with the highest CGPA.

```
1 print("\n--- Top 5 students with the highest CGPA ---")
2 display(df.sort_values(by='CGPA', ascending=False).head(5))
```

Output

Students sorted by height in descending order											
ID	Age	Height	Weight	Gender	Department	Year	Semester	MatrId	Wt. Used	Wt.Used/A	Performance
940	200	22	188.88	86.77	Female	IT	2018	0	Yes	11.001278	Good
541	224	18	188.82	43.78	Male	IT	2018	3	Yes	17.718077	Average
542	181	18	188.68	87.18	Female	IS	2018	0	Yes	11.200777	Average
688	720	27	188.67	73.82	Female	IS	2018	0	No	10.000000	Good
222	280	21	188.75	94.24	Female	IT	2018	7	No	21.019500	Average

98	20	20	188.18	91.01	Male	CS	2018	0	No	20.201000	Good
452	160	14	188.18	91.25	Male	IT	2018	0	Yes	20.240000	Good
200	448	19	188.17	83.88	Female	IS	2018	7	Yes	21.400000	Good
979	620	19	188.11	72.42	Female	IS	2018	0	Yes	21.207000	Yell
988	800	20	188.01	94.18	Male	CS	2018	0	Yes	21.242421	Good
878 rows x 12 columns											

Part H: Saving, Loading and Missing Values

1. Save the DataFrame into a CSV file named students.csv.

```
1 print("\n--- Saving DataFrame to students.csv ---")
2 df.to_csv('students.csv', index=False)
3 print("DataFrame successfully saved to 'students.csv'.")
```

Output

```
--- Saving DataFrame to students.csv ---
DataFrame successfully saved to 'students.csv'.
```

2. Read the CSV file into another DataFrame.

```
1 print("\n--- Reading students.csv into df_loaded ---")
2 df_loaded = pd.read_csv('students.csv')
3 print("DataFrame 'df_loaded' created from 'students.csv'.")
4 display(df_loaded.head())
5
6 print(f"\nShape of df_loaded: {df_loaded.shape}")
7 print("Info of df_loaded:")
8 df_loaded.info()
```

Output



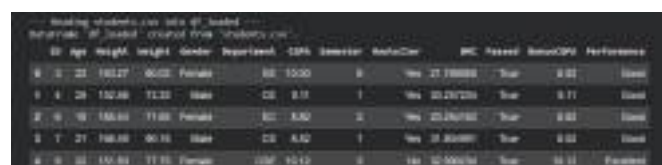
The screenshot shows the output of the `df_loaded.head()` command. It displays a table with 12 columns: `id`, `age`, `height`, `weight`, `gender`, `superhero`, `island`, `island`, `island`, `island`, `island`, `island`, and `Performance`. The first five rows are shown, with the first row having an `id` of 10 and a `Performance` of 10.00.

id	age	height	weight	gender	superhero	island	island	island	island	island	island	Performance
10	20	180.00	60.00	Male	10	10.00	1	Yes	10.00000	Yes	10.00	Excellent
100	40	20	40.00	Male	10	10.00	1	No	10.00000	Yes	10.00	Excellent
100	30	20	40.00	Female	10	10.00	1	Yes	10.00000	Yes	10.00	Excellent
10	40	20	40.00	Female	10	10.00	1	No	10.00000	Yes	10.00	Excellent
100	30	20	40.00	Male	10	10.00	1	Yes	10.00000	Yes	10.00	Excellent

3. Display the first five rows of the new DataFrame.

```
1 print("\n--- First five rows of the loaded DataFrame (df_loaded.head()) ←  
   ---")
2 display(df_loaded.head())
```

Output



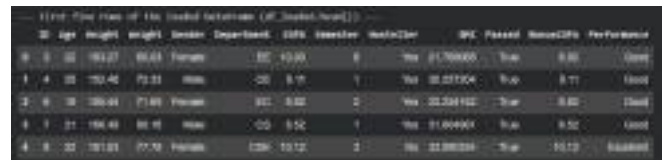
The screenshot shows the output of the `df_loaded.head()` command. It displays a table with 12 columns: `id`, `age`, `height`, `weight`, `gender`, `superhero`, `island`, `island`, `island`, `island`, `island`, `island`, and `Performance`. The first five rows are shown, with the first row having an `id` of 10 and a `Performance` of 10.00.

id	age	height	weight	gender	superhero	island	island	island	island	island	island	Performance
10	20	180.00	60.00	Male	10	10.00	1	Yes	10.00000	Yes	10.00	Excellent
100	40	20	40.00	Male	10	10.00	1	No	10.00000	Yes	10.00	Excellent
100	30	20	40.00	Female	10	10.00	1	Yes	10.00000	Yes	10.00	Excellent
10	40	20	40.00	Female	10	10.00	1	No	10.00000	Yes	10.00	Excellent
100	30	20	40.00	Male	10	10.00	1	Yes	10.00000	Yes	10.00	Excellent

4. Count the number of missing values in each column.

```
1 print("\n--- Counting missing values in each column of df_loaded ---")
2 display(df_loaded.isnull().sum())
```

Output



ID	Age	Height	Weight	Gender	Department	CGPA	Semester	Hosteller	BMI	Passed	BonusCGPA	Performance
0	20	161.27	65.64	Female	EE	75.80	8	Yes	21.766665	True	8.80	Good
1	14	150.46	73.31	Male	CE	8.95	1	No	30.23304	False	8.71	Good
2	19	156.44	71.49	Female	EE	8.80	2	Yes	22.54192	True	8.80	Good
3	1	156.48	66.16	Male	CE	5.52	1	No	21.86494	False	8.52	Good
4	18	161.81	75.76	Female	CE	70.12	2	No	22.88234	True	10.12	Excellent

5. Remove all rows containing missing values.

```
1 print("\n--- Removing rows with missing values from df_loaded ---")
2 print(f"Shape of df_loaded before dropping NaNs: {df_loaded.shape}")
3
4 df_loaded = df_loaded.dropna()
5
6 print(f"Shape of df_loaded after dropping NaNs: {df_loaded.shape}")
7 print("DataFrame after removing rows with missing values (first 5 rows):"↵
8       )
9 display(df_loaded.head())
```

Output



ID	0
Age	0
Height	0
Weight	0
Gender	0
Department	1
CGPA	0
Semester	0
Hosteller	0
BMI	0
Passed	0
BonusCGPA	0
Performance	0

dtype: int64

6. Replace all missing values with 0.

```
1 print("\n--- Replacing all missing values with 0 in df_loaded ---")
2 df_loaded = df_loaded.fillna(0)
3
4 print("DataFrame after filling NaNs with 0 (first 5 rows):")
5 display(df_loaded.head())
6
7 print("\n--- Counting missing values in each column of df_loaded after ←
      filling ---")
8 display(df_loaded.isnull().sum())
```

Output

Replaced rows with missing values from df_loaded :-
Shape of df_loaded before dropping nans : (204, 13)
Shape of df_loaded after dropping nans : (202, 13)
DataFrame after replacing rows with missing values (first 5 rows):

	id	Age	height	weight	gender	department	QPM	speaker	modulator	all	Passed	ResultQPM	Performance
0	3	22	162.27	60.03	Female	SD	92.00	0	Yes	21.706008	Yes	8.02	Good
1	4	28	162.46	52.83	Male	CD	9.11	1	Yes	30.287299	Yes	8.19	Good
2	8	19	165.06	71.88	Female	SD	8.88	0	Yes	30.204182	Yes	8.80	Good
3	5	21	166.19	66.16	Male	CD	8.03	1	Yes	21.694991	Yes	9.00	Good
4	8	22	165.33	71.79	Female	CD	90.13	0	No	32.860234	Yes	10.13	Excellent

Assignment 4

Exploratory Data Analysis (EDA), Data Preprocessing, and Visualization

Implement the following computations using Pandas, Seaborn, Matplotlib, and Tensor-Flow.

1. Load the dataset and import it into a Pandas DataFrame. Use the dataset.

```
1 import pandas as pd
2
3 # Google Sheet URL (ensure it's shareable and publicly accessible or that you ↵
   are logged into a Google account with access)
4 google_sheet_url = "https://docs.google.com/spreadsheets/d/1↵
   HgdAx_pUi8gzdBnk203EZlh-90cmhdLTpHeWmxQpQo/edit?usp=sharing"
5
6 # Convert the Google Sheet URL to a CSV export URL
7 csv_export_url = google_sheet_url.replace("/edit?usp=sharing", "/export?format↵
   =csv")
8
9 # Load the dataset into a Pandas DataFrame
10 try:
11     df = pd.read_csv(csv_export_url)
12     print("Dataset loaded successfully!")
13     print(df.head())
14 except Exception as e:
15     print(f"Error loading dataset: {e}")
16     print("Please ensure the Google Sheet is published to the web or set to '↵
   Anyone with the link can view' to allow access.")
```

Output

Dataset loaded successfully!

	X1 transaction date	X2 house age	X3 distance to the nearest MRT station \
0	2012.917	32.0	84.87882
1	2012.917	19.5	306.59470
2	2013.583	13.3	561.98450
3	2013.500	13.3	561.98450
4	2012.833	5.0	390.56840

	X4 number of convenience stores	X5 latitude	X6 longitude \
--	---------------------------------	-------------	----------------

0	10.0	24.98298	121.54024
1	9.0	24.98034	121.53951
2	5.0	24.98746	121.54391
3	5.0	24.98746	121.54391
4	5.0	24.97937	121.54245

Y house price of unit area

0	37.9
1	42.2
2	47.3
3	54.8
4	43.1

2. Display the first five rows and the last three rows of the dataset.

```
1 print("First five rows of the dataset:")
2 print(df.head(5))
3
4 print("\nLast three rows of the dataset:")
5 print(df.tail(3))
```

Output

First five rows of the dataset:

	X1 transaction date	X2 house age	X3 distance to the nearest MRT station	\
0	2012.917	32.0		84.87882
1	2012.917	19.5		306.59470
2	2013.583	13.3		561.98450
3	2013.500	13.3		561.98450
4	2012.833	5.0		390.56840

	X4 number of convenience stores	X5 latitude	X6 longitude	\
0		10.0	24.98298	121.54024
1		9.0	24.98034	121.53951
2		5.0	24.98746	121.54391
3		5.0	24.98746	121.54391
4		5.0	24.97937	121.54245

Y house price of unit area

0	37.9
1	42.2
2	47.3
3	54.8
4	43.1

Last three rows of the dataset:

	X1 transaction date	X2 house age	\
412	2013.000	8.1	
413	2013.500	6.5	
414	2013.167	1.9	

	X3 distance to the nearest MRT station	X4 number of convenience stores	\
412	104.81010		5.0
413	90.45606		9.0
414	355.00000		NaN

	X5 latitude	X6 longitude	Y house price of unit area
412	24.96674	121.54067	52.5
413	24.97433	121.54310	63.9
414	24.97293	121.54026	40.5

3. Get the dimensions (number of rows and columns) of the dataset.

```
1 print(f"The dataset has {df.shape[0]} rows and {df.shape[1]} columns.")
```

Output

The dataset has 415 rows and 7 columns.

4. Display the names of all columns (attributes).

```
1 print("Column names:")
2 print(df.columns)
```

Output

Column names:

```
Index(['X1 transaction date', 'X2 house age',
       'X3 distance to the nearest MRT station',
       'X4 number of convenience stores', 'X5 latitude', 'X6 longitude',
       'Y house price of unit area'],
      dtype='object')
```

5. Generate descriptive statistics (mean, median, standard deviation, variance, five-point summary, IQR, etc.) for the numerical attributes.

```
1 print("Descriptive statistics for numerical attributes:")
2 print(df.describe())
```

Output

Descriptive statistics for numerical attributes:

	X1 transaction date	X2 house age \
count	415.000000	415.000000
mean	2013.149014	17.674458
std	0.281628	11.405161
min	2012.667000	0.000000
25%	2012.917000	8.950000
50%	2013.167000	16.100000
75%	2013.417000	28.100000
max	2013.583000	43.800000

	X3 distance to the nearest MRT station \
count	415.000000
mean	1082.129338
std	1261.092057
min	23.382840
25%	289.324800
50%	492.231300
75%	1452.760000
max	6488.021000

	X4 number of convenience stores	X5 latitude	X6 longitude \
count	414.000000	415.000000	415.000000
mean	4.094203	24.969039	121.533378
std	2.945562	0.012397	0.015332
min	0.000000	24.932070	121.473530
25%	1.000000	24.963010	121.528570
50%	4.000000	24.971100	121.538630
75%	6.000000	24.977450	121.543300
max	10.000000	25.014590	121.566270

	Y house price of unit area
count	415.000000
mean	37.986265
std	13.590608
min	7.600000
25%	27.700000
50%	38.500000
75%	46.600000
max	117.500000

6. Print a concise summary of the dataset, including the data types (schema) and the number of missing values in each column.

```
1 print("\nDataset Information (Data Types and Non-Null Counts):")
2 df.info()
3
4 print("\nNumber of Missing Values Per Column:")
5 print(df.isnull().sum())
```

Output

Dataset Information (Data Types and Non-Null Counts):

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 415 entries, 0 to 414

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	X1 transaction date	415 non-null	float64
1	X2 house age	415 non-null	float64
2	X3 distance to the nearest MRT station	415 non-null	float64
3	X4 number of convenience stores	414 non-null	float64
4	X5 latitude	415 non-null	float64
5	X6 longitude	415 non-null	float64
6	Y house price of unit area	415 non-null	float64

dtypes: float64(7)

memory usage: 22.8 KB

Number of Missing Values Per Column:

X1 transaction date	0
X2 house age	0
X3 distance to the nearest MRT station	0
X4 number of convenience stores	1
X5 latitude	0
X6 longitude	0
Y house price of unit area	0

dtype: int64

7. Add a new column named “X22” by converting the “house age” from years to days.

```
1 df['X22'] = df['X2 house age'] * 365
2 print("Added 'X22' column (house age in days). Here are the first 5 rows with ←
    the new column:")
3 print(df[['X2 house age', 'X22']].head())
```

Output

Added 'X22' column (house age in days). Here are the first 5 rows with the new column:

	X2 house age	X22
0	32.0	11680.0
1	19.5	7117.5
2	13.3	4854.5
3	13.3	4854.5
4	5.0	1825.0

8. Delete the column “X22” from the dataset.

```
1 df = df.drop('X22', axis=1)
2 print("Column 'X22' deleted. Here are the first 5 rows of the DataFrame:")
3 print(df.head())
```

Output

Column 'X22' deleted. Here are the first 5 rows of the DataFrame:

	X1 transaction date	X2 house age	X3 distance to the nearest MRT station \
0	2012.917	32.0	84.87882
1	2012.917	19.5	306.59470
2	2013.583	13.3	561.98450
3	2013.500	13.3	561.98450
4	2012.833	5.0	390.56840

	X4 number of convenience stores	X5 latitude	X6 longitude \
0	10.0	24.98298	121.54024
1	9.0	24.98034	121.53951
2	5.0	24.98746	121.54391
3	5.0	24.98746	121.54391
4	5.0	24.97937	121.54245

	Y house price of unit area
0	37.9
1	42.2
2	47.3
3	54.8
4	43.1

9. Create three new instances synthetically and append them to the dataset.

```
1 import pandas as pd
2
3 # Create three new synthetic instances
4 new_instances_data = {
5     'X1 transaction date': [2013.25, 2013.75, 2012.75],
6     'X2 house age': [15.0, 5.0, 30.0],
7     'X3 distance to the nearest MRT station': [250.0, 1500.0, 600.0],
8     'X4 number of convenience stores': [8.0, 2.0, 5.0],
9     'X5 latitude': [24.98000, 24.97000, 24.99000],
10    'X6 longitude': [121.54000, 121.53000, 121.55000],
11    'Y house price of unit area': [45.0, 30.0, 50.0]
12 }
13
14 new_instances_df = pd.DataFrame(new_instances_data)
15
16 # Append the new instances to the original DataFrame
17 df = pd.concat([df, new_instances_df], ignore_index=True)
18
19 print("Three new synthetic instances have been added to the dataset.")
20 print("Displaying the tail of the DataFrame to show the new instances:")
21 print(df.tail())
```

Output

Three new synthetic instances have been added to the dataset.
Displaying the tail of the DataFrame to show the new instances:

	X1 transaction date	X2 house age	\
413	2013.500	6.5	
414	2013.167	1.9	
415	2013.250	15.0	
416	2013.750	5.0	
417	2012.750	30.0	

	X3 distance to the nearest MRT station	X4 number of convenience stores	\
413	90.45606	9.0	
414	355.00000	NaN	
415	250.00000	8.0	
416	1500.00000	2.0	
417	600.00000	5.0	

	X5 latitude	X6 longitude	Y house price of unit area
413	24.97433	121.54310	63.9
414	24.97293	121.54026	40.5
415	24.98000	121.54000	45.0
416	24.97000	121.53000	30.0
417	24.99000	121.55000	50.0

10. Delete the newly inserted three instances.

```
1 df = df.iloc[:-3]
2 print("The three newly inserted instances have been deleted.")
3 print("Displaying the tail of the DataFrame to confirm the removal:")
4 print(df.tail())
```

Output

The three newly inserted instances have been deleted.

Displaying the tail of the DataFrame to confirm the removal:

	X1 transaction date	X2 house age \		X3 distance to the nearest MRT station	X4 number of convenience stores \
410	2012.667	5.6		90.45606	9.0
411	2013.250	18.8		390.96960	7.0
412	2013.000	8.1		104.81010	5.0
413	2013.500	6.5		90.45606	9.0
414	2013.167	1.9		355.00000	NaN

	X5 latitude	X6 longitude	Y house price of unit area
410	24.97433	121.54310	50.0
411	24.97923	121.53986	40.6
412	24.96674	121.54067	52.5
413	24.97433	121.54310	63.9
414	24.97293	121.54026	40.5

11. Update the “house price of unit area” to 110 wherever the current value is greater than 110.

```
1 print("Original maximum house price of unit area:", df['Y house price of unit ↵
   area'].max())
2
3 # Update 'Y house price of unit area' to 110 wherever the current value is ↵
   greater than 110
4 df.loc[df['Y house price of unit area'] > 110, 'Y house price of unit area'] =↵
   110
5
6 print("Updated maximum house price of unit area:", df['Y house price of unit ↵
   area'].max())
7 print("\nDataFrame head after updating house prices:")
8 print(df.head())
```

Output

Original maximum house price of unit area: 117.5

Updated maximum house price of unit area: 110.0

DataFrame head after updating house prices:

	X1 transaction date	X2 house age	X3 distance to the nearest MRT station	\
0	2012.917	32.0		84.87882
1	2012.917	19.5		306.59470
2	2013.583	13.3		561.98450
3	2013.500	13.3		561.98450
4	2012.833	5.0		390.56840

	X4 number of convenience stores	X5 latitude	X6 longitude	\
0	10.0	24.98298	121.54024	
1	9.0	24.98034	121.53951	
2	5.0	24.98746	121.54391	
3	5.0	24.98746	121.54391	
4	5.0	24.97937	121.54245	

	Y house price of unit area
0	37.9
1	42.2
2	47.3
3	54.8
4	43.1

12. Find the latitude and longitude of the houses whose prices are less than or equal to 20.

```
1 print("Houses with 'Y house price of unit area' <= 20:")
2 houses_low_price = df[df['Y house price of unit area'] <= 20]
3 print(houses_low_price[['X5 latitude', 'X6 longitude']])
```

Output

Houses with 'Y house price of unit area' <= 20:

	X5 latitude	X6 longitude
8	24.95095	121.48458
40	24.94155	121.50381
41	24.94297	121.50342
48	24.94684	121.49578
49	24.94925	121.49542
55	24.94968	121.53009
73	24.94155	121.50381
83	24.96056	121.50831
87	24.94297	121.50342
93	24.94920	121.53076
113	24.96172	121.53812
116	24.94375	121.47883
117	24.93885	121.50383
155	24.94155	121.50381
156	24.94883	121.52954
162	24.94297	121.50342
170	24.94741	121.49628
176	24.94867	121.49507
180	24.94898	121.49621
183	24.94155	121.50381
226	24.94155	121.50381
229	24.94890	121.53095
231	24.94235	121.50357
232	24.95032	121.49587
249	24.95743	121.47516
251	24.94960	121.53018
255	24.95095	121.48458
298	24.94155	121.50381
309	24.94883	121.52954
320	24.93885	121.50383
329	24.93885	121.50383
330	24.94935	121.53046
331	24.94826	121.49587
347	24.95719	121.47353
384	24.94297	121.50342
409	24.94155	121.50381

13. Replace the missing values of the “number of convenience stores” attribute with the average number of convenience stores.

```
1 print("Number of missing values in 'X4 number of convenience stores' before ↵
    imputation:")
2 print(df['X4 number of convenience stores'].isnull().sum())
3
4 # Calculate the average of 'X4 number of convenience stores'
5 average_convenience_stores = df['X4 number of convenience stores'].mean()
6
7 # Replace missing values with the average
8 df['X4 number of convenience stores'].fillna(average_convenience_stores, ↵
    inplace=True)
9
10 print("\nNumber of missing values in 'X4 number of convenience stores' after ↵
    imputation:")
11 print(df['X4 number of convenience stores'].isnull().sum())
12
13 print("\nDataFrame info after imputation:")
14 df.info()
```

Output

Number of missing values in 'X4 number of convenience stores' before imputation:
1

Number of missing values in 'X4 number of convenience stores' after imputation:
0

DataFrame info after imputation:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 415 entries, 0 to 414

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	X1 transaction date	415 non-null	float64
1	X2 house age	415 non-null	float64
2	X3 distance to the nearest MRT station	415 non-null	float64
3	X4 number of convenience stores	415 non-null	float64
4	X5 latitude	415 non-null	float64
5	X6 longitude	415 non-null	float64
6	Y house price of unit area	415 non-null	float64

dtypes: float64(7)

14. Normalize the “distance to the nearest MRT station” using:

- (a) Z-score normalization.
- (b) Min-max normalization.
- (c) Decimal scaling.

```
1 import numpy as np
2
3 # Original column
4 distance_col = df['X3 distance to the nearest MRT station']
5
6 print("Original 'X3 distance to the nearest MRT station' (first 5 rows):"↵
      )
7 print(distance_col.head())
8 print("\n")
9
10 # (a) Z-score normalization
11 mean_distance = distance_col.mean()
12 std_distance = distance_col.std()
13 df['X3_distance_zscore'] = (distance_col - mean_distance) / std_distance
14 print("Z-score Normalized 'X3 distance to the nearest MRT station' (first↵
      5 rows):")
15 print(df['X3_distance_zscore'].head())
16 print("Mean: ", df['X3_distance_zscore'].mean())
17 print("Standard Deviation: ", df['X3_distance_zscore'].std())
18 print("\n")
19
20 # (b) Min-Max normalization
21 min_distance = distance_col.min()
22 max_distance = distance_col.max()
23 df['X3_distance_minmax'] = (distance_col - min_distance) / (max_distance ↵
      - min_distance)
24 print("Min-Max Normalized 'X3 distance to the nearest MRT station' (first↵
      5 rows):")
25 print(df['X3_distance_minmax'].head())
26 print("Min: ", df['X3_distance_minmax'].min())
27 print("Max: ", df['X3_distance_minmax'].max())
28 print("\n")
29
30 # (c) Decimal scaling
31 # Find the maximum absolute value to determine 'j'
32 max_abs_distance = abs(distance_col).max()
33 j = len(str(int(max_abs_distance))) if max_abs_distance > 0 else 1 # j is↵
      the number of digits
34
35 df['X3_distance_decimal'] = distance_col / (10**j)
36 print("Decimal Scaled 'X3 distance to the nearest MRT station' (first 5 ↵
      rows):")
37 print(df['X3_distance_decimal'].head())
```

```

38 print("Max Absolute Value: ", df['X3_distance_decimal'].abs().max())
39
40 # Displaying the new columns along with the original for comparison
41 print("\nComparison of original and normalized columns (first 5 rows):")
42 print(df[['X3 distance to the nearest MRT station', 'X3_distance_zscore', ←
           'X3_distance_minmax', 'X3_distance_decimal']].head())

```

Output

Original 'X3 distance to the nearest MRT station' (first 5 rows):

```

0    84.87882
1    306.59470
2    561.98450
3    561.98450
4    390.56840

```

Name: X3 distance to the nearest MRT station, dtype: float64

Z-score Normalized 'X3 distance to the nearest MRT station' (first 5 rows):

```

0    -0.790783
1    -0.614971
2    -0.412456
3    -0.412456
4    -0.548383

```

Name: X3_distance_zscore, dtype: float64

Mean: 4.280377926265664e-18

Standard Deviation: 1.0000000000000002

Min-Max Normalized 'X3 distance to the nearest MRT station' (first 5 rows):

```

0    0.009513
1    0.043809
2    0.083315
3    0.083315
4    0.056799

```

Name: X3_distance_minmax, dtype: float64

Min: 0.0

Max: 1.0

Decimal Scaled 'X3 distance to the nearest MRT station' (first 5 rows):

```

0    0.008488
1    0.030659
2    0.056198
3    0.056198
4    0.039057

```

Name: X3_distance_decimal, dtype: float64

Max Absolute Value: 0.6488020999999999

Comparison of original and normalized columns (first 5 rows):

	X3 distance to the nearest MRT station	X3_distance_zscore \
0	84.87882	-0.790783
1	306.59470	-0.614971
2	561.98450	-0.412456
3	561.98450	-0.412456
4	390.56840	-0.548383

	X3_distance_minmax	X3_distance_decimal
0	0.009513	0.008488
1	0.043809	0.030659
2	0.083315	0.056198
3	0.083315	0.056198
4	0.056799	0.039057

15. Generate the following visualizations using Seaborn/Matplotlib. Add suitable titles, axis labels, legends, and color schemes.
- (a) Create a histogram of the “Y house price of unit area” attribute.
 - (b) Create a box-and-whisker plot for the “Y house price of unit area” attribute.
 - (c) Create a scatter plot of house price versus house age.
 - (d) Create another scatter plot of house price versus distance to the nearest MRT station.
 - (e) Draw a correlation heatmap for all numerical attributes.
 - (f) Create a pair plot (scatter matrix) for the numerical attributes.
 - (g) Plot the distribution (KDE or histogram with KDE) of the house price.
 - (h) Create a count plot showing the distribution of the number of convenience stores.
 - (i) Create a bar chart showing the average house price for each number of convenience stores.
 - (j) Draw a line plot showing the relationship between house age and average house price.

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 plt.style.use('seaborn-v0_8-darkgrid')
5
6 # Set up figure size for better readability
7 plt.figure(figsize=(10, 6))
8
9 # (a) Create a histogram of the Y house price of unit area ↵
   attribute.
10 sns.histplot(df['Y house price of unit area'], bins=30, kde=True, color='↵
   skyblue')
11 plt.title('Distribution of House Price of Unit Area')
12 plt.xlabel('House Price of Unit Area')
13 plt.ylabel('Frequency')
14 plt.show()
15
16 # (b) Create a box-and-whisker plot for the Y house price of unit ↵
   area attribute.
17 plt.figure(figsize=(10, 6))
18 sns.boxplot(y=df['Y house price of unit area'], color='lightcoral')
19 plt.title('Box Plot of House Price of Unit Area')
20 plt.ylabel('House Price of Unit Area')
21 plt.show()
22
23 # (c) Create a scatter plot of house price versus house age.
24 plt.figure(figsize=(10, 6))
25 sns.scatterplot(x='X2 house age', y='Y house price of unit area', data=df↵
   , hue='Y house price of unit area', palette='viridis')
26 plt.title('House Price vs. House Age')
27 plt.xlabel('House Age (years)')
28 plt.ylabel('House Price of Unit Area')
29 plt.legend(title='Price')
```

```

30 plt.show()
31
32 # (d) Create another scatter plot of house price versus distance to the ←
    nearest MRT station.
33 plt.figure(figsize=(10, 6))
34 sns.scatterplot(x='X3 distance to the nearest MRT station', y='Y house ←
    price of unit area', data=df, hue='Y house price of unit area', ←
    palette='magma')
35 plt.title('House Price vs. Distance to Nearest MRT Station')
36 plt.xlabel('Distance to Nearest MRT Station')
37 plt.ylabel('House Price of Unit Area')
38 plt.legend(title='Price')
39 plt.show()
40
41 # (e) Draw a correlation heatmap for all numerical attributes.
42 plt.figure(figsize=(12, 10))
43 correlation_matrix = df.drop(columns=['X3_distance_zscore', '←
    X3_distance_minmax', 'X3_distance_decimal'], errors='ignore').corr()
44 sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', ←
    linewidths=.5)
45 plt.title('Correlation Heatmap of Numerical Attributes')
46 plt.show()
47
48 # (f) Create a pair plot (scatter matrix) for the numerical attributes.
49 # Selecting a subset of original numerical columns for clarity in pair ←
    plot
50 numerical_cols = ['X1 transaction date', 'X2 house age', 'X3 distance to ←
    the nearest MRT station',
51                  'X4 number of convenience stores', 'Y house price of ←
    unit area']
52 sns.pairplot(df[numerical_cols], diag_kind='kde')
53 plt.suptitle('Pair Plot of Numerical Attributes', y=1.02) # Adjust y to ←
    prevent title overlap
54 plt.show()
55
56 # (g) Plot the distribution (KDE or histogram with KDE) of the house ←
    price.
57 plt.figure(figsize=(10, 6))
58 sns.histplot(df['Y house price of unit area'], kde=True, color='purple', ←
    bins=30)
59 plt.title('Distribution of House Price of Unit Area (Histogram with KDE)'←
    )
60 plt.xlabel('House Price of Unit Area')
61 plt.ylabel('Density/Frequency')
62 plt.show()
63
64 # (h) Create a count plot showing the distribution of the number of ←
    convenience stores.
65 plt.figure(figsize=(10, 6))

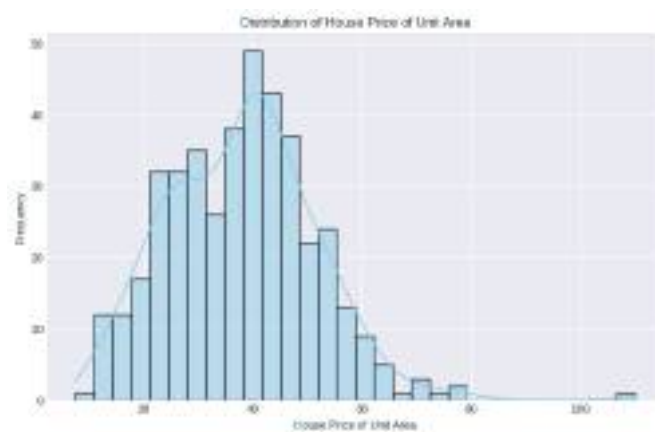
```

```

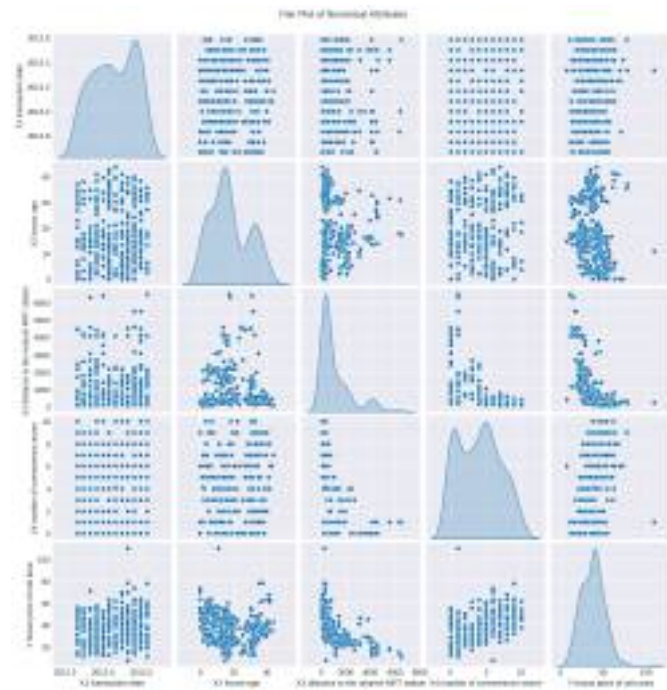
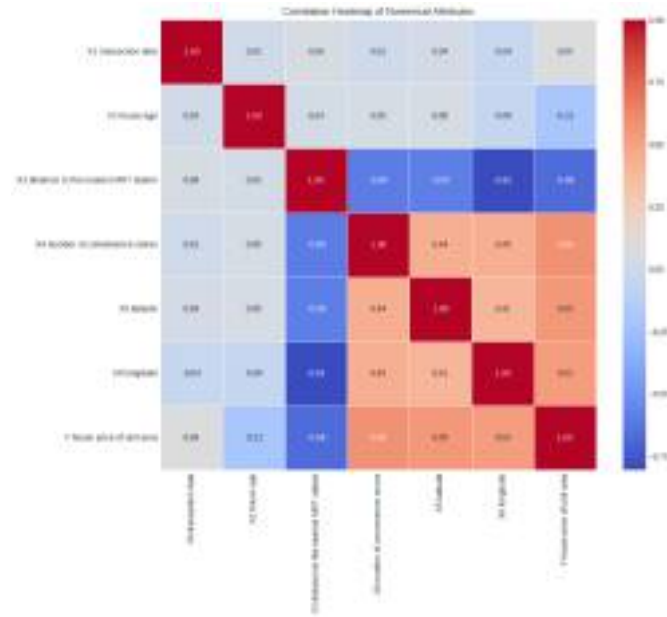
66 sns.countplot(x='X4 number of convenience stores', data=df, palette='viridis')
67 plt.title('Distribution of Number of Convenience Stores')
68 plt.xlabel('Number of Convenience Stores')
69 plt.ylabel('Count')
70 plt.show()
71
72 # (i) Create a bar chart showing the average house price for each number of convenience stores.
73 plt.figure(figsize=(10, 6))
74 average_price_by_stores = df.groupby('X4 number of convenience stores')['Y house price of unit area'].mean().reset_index()
75 sns.barplot(x='X4 number of convenience stores', y='Y house price of unit area', data=average_price_by_stores, palette='Blues_d')
76 plt.title('Average House Price by Number of Convenience Stores')
77 plt.xlabel('Number of Convenience Stores')
78 plt.ylabel('Average House Price of Unit Area')
79 plt.show()
80
81 # (j) Draw a line plot showing the relationship between house age and average house price.
82 plt.figure(figsize=(12, 6))
83 # Group by house age and calculate mean price, then sort by house age for a clear line plot
84 average_price_by_age = df.groupby('X2 house age')['Y house price of unit area'].mean().reset_index()
85 average_price_by_age = average_price_by_age.sort_values('X2 house age')
86 sns.lineplot(x='X2 house age', y='Y house price of unit area', data=average_price_by_age, marker='o', color='green')
87 plt.title('Relationship Between House Age and Average House Price')
88 plt.xlabel('House Age (years)')
89 plt.ylabel('Average House Price of Unit Area')
90 plt.grid(True)
91 plt.show()

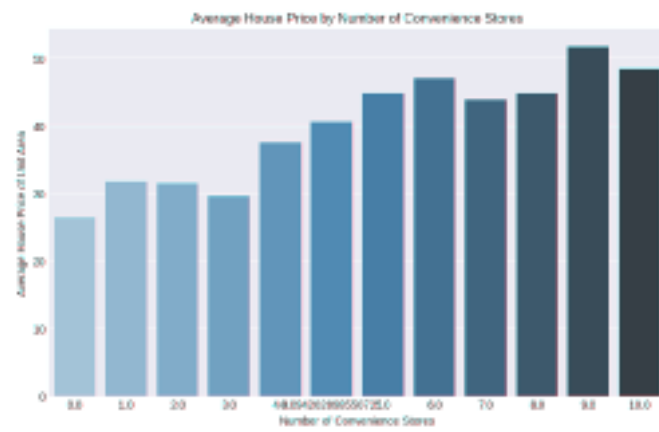
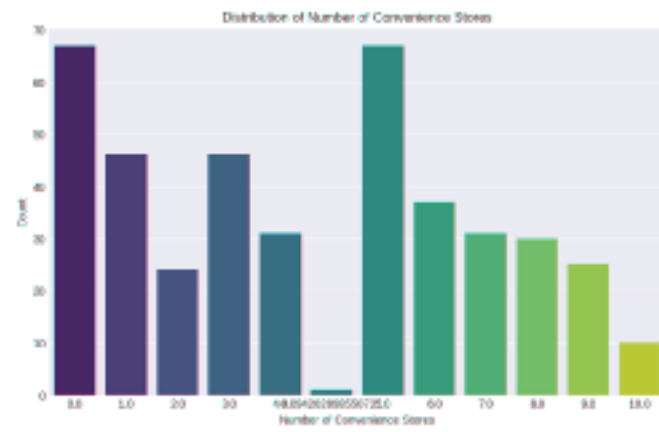
```

Output











16. Based on the exploratory analysis and visualizations, identify:

- (a) The attribute that has the strongest positive correlation with house price.
- (b) The attribute that has the strongest negative correlation with house price.
- (c) Any possible outliers in the dataset.

Based on the exploratory analysis and visualizations:

(a) The attribute that has the strongest positive correlation with house price: Looking at the correlation heatmap, 'X4 number of convenience stores' has the strongest positive correlation with 'Y house price of unit area' at approximately **0.57**. This suggests that houses closer to more convenience stores tend to have higher unit prices.

(b) The attribute that has the strongest negative correlation with house price: From the correlation heatmap, 'X3 distance to the nearest MRT station' has the strongest negative correlation with 'Y house price of unit area' at approximately **-0.67**. This indicates that as the distance to the nearest MRT station increases, the house price tends to decrease.

(c) Any possible outliers in the dataset: **House Price:** The box-and-whisker plot for 'Y house price of unit area' clearly shows several data points above the upper whisker, indicating potential outliers with unusually high house prices, even after capping the maximum at 110.0. The maximum value in the box plot is around 110, which corresponds to the capping performed earlier, but values around the 80-110 range appear as distinct points above the main distribution. The scatter plots also show some points with high house prices that are somewhat separated from the main clusters. **Distance to MRT Station:** The scatter plot of 'House Price vs. Distance to Nearest MRT Station' shows some houses with very high distances to the MRT station (e.g., above 4000 meters) that still have moderate house prices, which might be considered outliers depending on further domain knowledge.

These outliers should be further investigated to determine if they are data entry errors, rare events, or genuinely exceptional cases that could provide valuable insights.